

Contents

1. Overview	2
2. Base System	2
2.1. Semantic Typing Definitions	2
2.2. Grammar	3
2.3. Rules	3
2.4. Soundness and Completeness of the Base System	7
3. Extensions	7
3.1. Records	7
3.2. Modules	9
3.3. Variants	11
3.4. Polymorphism	12
3.5. Recursion	13
3.6. Recursive types	14
3.7. Refinement types	16
3.8. Dependent functions	17
3.9. Singleton types	20
3.10. Unit	20
3.11. Bottom	21
3.12. Lists	22
4. Type splaying	22
4.1. Splaying Recursive Functions	22
4.2. Splaying Recursive Types	22
5. Wrap	25
5.1. Base wrap	25
5.2. Record wrapping	28
5.3. Module wrapping	28
5.4. Dependent function wrapping	29
5.5. Other wrappers	30
6. Concolic	31
6.1. Concolic Steps	31
6.1.1. Concolic values	32
6.1.2. Timestamps	33
6.1.3. Input environments	34
6.1.4. Labels in input environments	35
6.1.5. Path logging	36
6.2. Base concolic system	37
6.2.1. Grammar	37
6.2.2. Rules	38
6.2.3. Concolic loop	41
6.2.4. Efficiency of concolic evaluation in practice	43
6.2.4.1. Solving labels	43
6.2.4.2. Solving formulas	43
6.2.4.3. Target prioritization	43
6.2.4.4. Continuation reuse	43
6.3. Concolic Extensions	44
6.3.1. Concolic Records	44

6.3.2. Concolic Modules	44
6.3.3. Concolic Variants	45
6.3.4. Concolic Refinement Types	46

1. Overview

In this document, we present a system for semantic type refutation. We begin with a set of nondeterministic rules over a simple, core language, and we slowly expand the language and rules to a full, useful, type checking system.

In Section 2 we present the base system. The base system is a set of nondeterministic inference rules to refute the semantic typing of a program in a simple language. It is not designed to be minimal in isolation but rather to be easily extensible; the base system could be condensed if it were never to be extended.

In Section 3 we extend the base language with features that make the language more useful. Features include records, variants, recursive functions, recursive types, the type of types, and more.

We then present in Section 4 a terminating variation of the system called “type splaying” that is incomplete with respect to semantic typing: it finds false refutations, but it will terminate on recursive programs and can thus prove recursive programs well typed.

In Section 5 is a notion of usage checking via “wrapping”. This way, usages of type-annotated values are checked to comply with such annotations.

Finally, in Section 6, we walk through a series of extensions that naturally transform the system into a concolic evaluator. It is then no longer oracular but rather is an algorithmic system to find type refutations or prove the lack thereof.

2. Base System

2.1. Semantic Typing Definitions

What is semantic typing? Why do semantic typing? What does it mean to type with refutation? We give a few informal intuitions in this section.

Semantic typing is an approach to type checking that is not syntactic, but it instead uses the semantic meaning of expressions and how they evaluate. We will use the model theory of types: types are sets of values. $\llbracket t \rrbracket$ is the set of values in the type t .

Assume we have a standard (currently undefined) evaluation relation $e \rightarrow^* v$ if e evaluates to value v and $e \rightarrow^* \text{ERROR}$ if e evaluates to an ERROR expression.

We say $\models e :: t$ if $e \nrightarrow^* \text{ERROR}$ and if $\forall v. e \rightarrow^* v$, then $v \in \llbracket t \rrbracket$. It remains to define $\llbracket t \rrbracket$ for each type t . To keep it simple, we will only work over simple types to begin. These are:

- $\llbracket \text{int} \rrbracket = \mathbb{Z}$.
- $\llbracket \text{bool} \rrbracket = \{\text{true}, \text{false}\}$.
- $\llbracket t_1 \rightarrow t_2 \rrbracket = \{v \mid \forall v'. v' \in \llbracket t_1 \rrbracket. \models v v' :: t_2\}$.

This means that a value has type $t_1 \rightarrow t_2$ if it evaluates (without erroring) to a value in type t_2 for every argument in type t_1 . Thus it is relatively easy to disprove that v is in type $t_1 \rightarrow t_2$: we simply need to find one value in t_1 for which the function application goes wrong. This is what we mean to find a refutation.

For example, consider $f = \text{fun } x \rightarrow \text{if } x = 0 \text{ then } 1 \text{ else true}$. We can *refute* that f has type $\text{int} \rightarrow \text{bool}$ with the following counterexample: $f\ 0 \rightarrow^* 1$, and 1 is not a boolean. This means the program $\text{let } f : \text{int} \rightarrow \text{bool} = \text{fun } x \rightarrow \text{if } x = 0 \text{ then } 1 \text{ else true}$ is not semantically well typed. If a refutation system is sound and complete with respect to the above definitions of typing, then if the system declares a program is ill typed, it really is! There is some way the program can run that violates the typing.

Consider a similar program: $\text{let } f : \text{int} \rightarrow \text{bool} = \text{fun } x \rightarrow \text{if false then } 1 \text{ else true}$. This program is semantically well typed, but a standard syntactic type checker would reject the program because the two branches have different types, even if one is dead code.

2.2. Grammar

The grammar describes a simple functional language but with a few oddities: types are values; there is a specific value for a generated member of a function type; evaluation can produce a value or a result of a type check.

variables	x	$::=$	$(Identifiers)$	
integers	n	$::=$	$0 \mid 1 \mid 2 \mid \dots$	
booleans	b	$::=$	$\text{true} \mid \text{false}$	
values	v	$::=$	n	
			b	
			$\langle \lambda x.e, E \rangle$	Function closure
			t	Type value
			g	Generated value
type values	t	$::=$	int	
			bool	
			$t \rightarrow t$	Function type value
generated values	g	$::=$	$G(t \rightarrow t)$	Generated function
expressions	e	$::=$	τ	Type as expression
			x	
			n	
			b	
			$\lambda x.e$	
			$e\ e$	
			$e \otimes e$	Arbitrary binary operation
			$\text{if } e \text{ then } e \text{ else } e$	
			$\text{let } x = e \text{ in } e$	Untyped let-expression
			$\text{let } x : \tau = e \text{ in } e$	Typed let-expression
type expressions	τ	$::=$	e	Expression as type
			int	
			bool	
			$\tau \rightarrow \tau$	Function type expression
binary operations	$\otimes$	$::=$	$+$ $ $ $-$ $ $ $*$ $ $ $/$ $ $ $==$	
type check result	$\odot$	$::=$	$\mathbf{X}$	Refutation
			$\checkmark$	Confirmation
evaluation result	r	$::=$	v	
			$\odot$	
environments	E	$::=$	$\{x \mapsto v, \dots, x \mapsto v\}$	

Figure 1: Core Grammar

2.3. Rules

The rules have three different judgements, each written using a distinct arrow. The judgements correspond to evaluating, generating, and checking. We explain informally before presenting the rules.

We write $E \models e \Rightarrow v$ for big-step evaluation within the semantic typing system.

$t \uparrow v$ means “type t generates value v ”, where v is a member of the type t . The upward arrow signals production; it constructs a value v .

$v : t \Downarrow \odot$ means “the result of checking that value v has type t is $\odot$ ”. Since checking is the complement to generating, we use the downward arrow for checking. The downward arrow signals destruction rather than production; it yields only a refutation or confirmation.

We first show the untyped evaluation rules. These are standard for environment-based big step semantics.

$E \models e \Rightarrow r$

 (1 of 5)
$$\begin{array}{c}
\frac{(x \mapsto v) \in E}{E \models x \Rightarrow v} \text{VARIABLE} \qquad \frac{}{E \models \lambda x. e \Rightarrow \langle \lambda x. e, E \rangle} \text{CLOSURE} \\
\\
\frac{E \models e_1 \Rightarrow v_1 \quad E \models e_2 \Rightarrow v_2}{E \models e_1 \otimes e_2 \Rightarrow v_1 \llbracket \otimes \rrbracket v_2} \text{BINOP} \\
\\
\frac{E \models e_1 \Rightarrow \langle \lambda x. e, E' \rangle \quad E \models e_2 \Rightarrow v \quad E' \{x \mapsto v\} \models e \Rightarrow r}{E \models e_1 \ e_2 \Rightarrow r} \text{APPL} \\
\\
\frac{E \models e_1 \Rightarrow \text{true} \quad E \models e_2 \Rightarrow r}{E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFTRUE} \\
\\
\frac{E \models e_1 \Rightarrow \text{false} \quad E \models e_3 \Rightarrow r}{E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFFALSE} \qquad \frac{E \models e_1 \Rightarrow v \quad E \{x \mapsto v\} \models e_2 \Rightarrow r}{E \models \text{let } x = e_1 \text{ in } e_2 \Rightarrow r} \text{LET}
\end{array}$$

Figure 2: Evaluate untyped expressions

Next are the rules to eagerly evaluate type expressions to type values.

$E \models e \Rightarrow r$

 (2 of 5)
$$\begin{array}{c}
\frac{}{E \models \text{int} \Rightarrow \text{int}} \text{INT} \qquad \frac{}{E \models \text{bool} \Rightarrow \text{bool}} \text{BOOL} \qquad \frac{E \models \tau_1 \Rightarrow t_1 \quad E \models \tau_2 \Rightarrow t_2}{E \models \tau_1 \rightarrow \tau_2 \Rightarrow t_1 \rightarrow t_2} \text{FUNTYPE}
\end{array}$$

Figure 3: Evaluate types

The generator rules are nondeterministic. They produce arbitrary members of the type. A generated function is a value with semantics allowing it to act like any real function; it is a value about which we can say “I don’t know what it does, but I know its type, so I can simulate interactions with it”.

$t \uparrow v$

$$\begin{array}{c}
\frac{b \in \mathbb{B}}{\text{bool} \uparrow b} \text{GENBOOL} \qquad \frac{n \in \mathbb{Z}}{\text{int} \uparrow n} \text{GENINT} \qquad \frac{}{t_1 \rightarrow t_2 \uparrow G(t_1 \rightarrow t_2)} \text{GENFUN}
\end{array}$$

Figure 4: Generators

Recall that $v : t \Downarrow \odot$ reads “the result of checking that value v has type t is $\odot$ ”. So $v : t \Downarrow \mathbf{X}$ means there is a refutation, and we therefore know $v \notin \llbracket t \rrbracket$. And $v : t \Downarrow \checkmark$ means there is a confirmation, which is just a failure to refute along one path. It takes the same form as a refutation, except the leaves are confirmations. While confirmations are not necessary for the correctness of this base system, they help when we extend it to a concolic system. Instead of having to say “the tree could not be built” about an attempted refutation, we would like to have some tree that represents the failure to refute.

$\boxed{v : t \Downarrow \odot}$ (1 of 2)

$$\begin{array}{c}
\frac{v \notin \mathbb{Z}}{v : \text{int} \Downarrow \mathbf{X}} \text{REFUTEINT} \quad \frac{v \in \mathbb{Z}}{v : \text{int} \Downarrow \checkmark} \text{CONFIRMIT} \quad \frac{v \notin \mathbb{B}}{v : \text{bool} \Downarrow \mathbf{X}} \text{REFUTEBOOL} \\
\\
\frac{v \in \mathbb{B}}{v : \text{bool} \Downarrow \checkmark} \text{CONFIRMBOL} \quad \frac{t_1 \Uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow v' \quad v' : t_2 \Downarrow \odot}{\langle \lambda x. e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKFUN} \\
\\
\frac{t_1 \Uparrow v \quad v : t'_1 \Downarrow \odot}{G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKGENFUNLEFT} \\
\\
\frac{t'_2 \Uparrow v \quad v : t_2 \Downarrow \odot}{G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKGENFUNRIGHT}
\end{array}$$

Figure 5: Checkers

We will now show how typed expressions can evaluate. Typed expressions can either evaluate regularly (i.e. evaluate the type and then throw it away) or be type checked.

$\boxed{E \models e \Rightarrow r}$ (3 of 5)

$$\begin{array}{c}
\frac{E \models e_1 \Rightarrow v \quad E \models \tau \Rightarrow t \quad E\{x \mapsto v\} \models e_2 \Rightarrow r}{E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow r} \text{LETYPED} \\
\\
\frac{E \models e_1 \Rightarrow v \quad E \models \tau \Rightarrow t \quad v : t \Downarrow \odot}{E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \odot} \text{LETCHECK} \\
\\
\frac{E \models e_1 \Rightarrow v \quad E \models \tau \Rightarrow v' \quad v' \text{ is not of the form } t}{E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \mathbf{X}} \text{LETFAILTYPEVAL}
\end{array}$$

Figure 6: Evaluate Typed Let-Expressions

The LETYPED and LETCHECK rules differ only in what they do after evaluating the let-definition and the type. It is a choice as to whether the body should be evaluated or the definition should be checked. When the definition is checked, the result of the type check is the result of the evaluation, where $\mathbf{X}$ means the evaluation errors, and $\checkmark$ means the evaluation vanishes. For example, this signals the evaluation errors:

$$\frac{E \models e_1 \Rightarrow v \quad E \models \tau \Rightarrow t \quad v : t \Downarrow \mathbf{X}}{E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \mathbf{X}}$$

In **LETFAILTYPEVAL**, we begin to see how a failure arises when an expression does not evaluate to the expected type. Failures also arise from runtime type mismatches, and we will use $E \models e \Rightarrow \mathbf{X}$ to signal a runtime type mismatch in e – it does not just mean there was a refutation from a checker. We cover the remaining such rules soon.

Applications to generated functions is similar to evaluating a typed **let**-expression: the argument can be checked, or a return value can be generated.

$E \models e \Rightarrow r$ (4 of 5)

$$\frac{E \models e_1 \Rightarrow G(t_1 \rightarrow t_2) \quad E \models e_2 \Rightarrow v \quad t_2 \uparrow v'}{E \models e_1 \ e_2 \Rightarrow v'} \text{APPLGENFUN}$$

$$\frac{E \models e_1 \Rightarrow G(t_1 \rightarrow t_2) \quad E \models e_2 \Rightarrow v \quad v : t_1 \Downarrow \odot}{E \models e_1 \ e_2 \Rightarrow \odot} \text{APPLCHECK}$$

Figure 7: Apply to Generated Functions

Runtime type mismatches arise in the following cases.

$E \models e \Rightarrow r$ (5 of 5)

$$\frac{E \models e_1 \Rightarrow v_1 \quad v_1 \text{ is not a valid operand with } \llbracket \otimes \rrbracket}{E \models e_1 \otimes e_2 \Rightarrow \mathbf{X}} \text{FAILBINOP1}$$

$$\frac{E \models e_1 \Rightarrow v_1 \quad E \models e_2 \Rightarrow v_2 \quad v_1 \text{ and } v_2 \text{ are not valid operands with } \llbracket \otimes \rrbracket}{E \models e_1 \otimes e_2 \Rightarrow \mathbf{X}} \text{FAILBINOP2}$$

$$\frac{E \models e_1 \Rightarrow v \quad v \in \mathbb{Z} \cup \mathbb{B} \text{ or } v \text{ is of the form } t}{E \models e_1 \ e_2 \Rightarrow \mathbf{X}} \text{FAILAPPL}$$

$$\frac{E \models e_1 \Rightarrow v \quad v \notin \mathbb{B}}{E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow \mathbf{X}} \text{FAILIF} \quad \frac{E \models \tau_1 \Rightarrow v \quad v \text{ is not of the form } t}{E \models \tau_1 \rightarrow \tau_2 \Rightarrow \mathbf{X}} \text{FAILFUNTYPE1}$$

$$\frac{E \models \tau_1 \Rightarrow t \quad E \models \tau_2 \Rightarrow v \quad v \text{ is not of the form } t}{E \models \tau_1 \rightarrow \tau_2 \Rightarrow \mathbf{X}} \text{FAILFUNTYPE2}$$

Figure 8: Type mismatches

When evaluating a function body when checking, non-evaluation of the body propagates:

$v : t \Downarrow \odot$ (2 of 2)

$$\frac{t_1 \uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow \odot}{\langle \lambda x. e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKFUNNONVALUE}$$

Figure 9: Checker via non-evaluation

The rules for propagation of non-evaluation are elided for brevity. Here are two examples, though, to demonstrate:

$$\frac{E \vdash e_1 \Rightarrow v \quad E \vdash e_2 \Rightarrow \odot}{E \vdash e_1 \otimes e_2 \Rightarrow \odot} \text{PROPAGATEBINOP}$$

$$\frac{E \vdash e_1 \Rightarrow \odot}{E \vdash e_1 \ e_2 \Rightarrow \odot} \text{PROPAGATEAPPL}$$

2.4. Soundness and Completeness of the Base System

The soundness claim of this system refers to a natural semantics written $E \vdash e \Rightarrow r$, during which types are evaluated and always thrown away. There is no refutation, confirmation, or generation of types in the natural semantics. Informally, if the evaluation of e contains any typed **let**-expression that is semantically ill-typed, then this double turnstile ($\Rightarrow$) system can error:

Claim 1 (Soundness): For expression e , we have $E \vdash e \Rightarrow \mathbf{X}$ if either

1. $\emptyset \vdash e \Rightarrow v$ has any proof tree

$$\frac{\frac{E \vdash e' \Rightarrow v' \quad E \vdash \tau \Rightarrow t \quad E\{x \mapsto v'\} \vdash e'' \Rightarrow v''}{E \vdash \text{let } x : \tau = e' \text{ in } e'' \Rightarrow v''}}{\dots}$$

$$\hline \emptyset \vdash e \Rightarrow v$$

where $v' \notin \llbracket t \rrbracket$, or

2. $\emptyset \vdash e \Rightarrow \mathbf{X}$.

The system is complete if the base language has a nondeterministic **pick_i** or **pick_b**. For now, we make no such statements about completeness.

3. Extensions

We now provide some type extensions on the base system. These new features are meant to make the language practical and expressive.

The features are presented as extensions, and it is noted explicitly where any previous rule is overwritten, and the rule uses the same name as one before.

Since this is a long section, we repeat the contents here. The content of this section is as follows.

3.1. Records	7
3.2. Modules	9
3.3. Variants	11
3.4. Polymorphism	12
3.5. Recursion	13
3.6. Recursive types	14
3.7. Refinement types	16
3.8. Dependent functions	17
3.9. Singleton types	20
3.10. Unit	20
3.11. Bottom	21
3.12. Lists	22

3.1. Records

We add records, which are mostly trivial. The nontrivialities are due to subtyping.

values	$v ::= \dots$	
	$\mid \{\ell = v; \dots; \ell = v\}$	Record value
expressions	$e ::= \dots$	
	$\mid \{\ell = e; \dots; \ell = e\}$	Record expression
	$\mid e.\ell$	Label projection
type values	$t ::= \dots$	
	$\mid \{\ell : t; \dots; \ell : t\}$	Record type value
types	$\tau ::= \dots$	
	$\mid \{\ell : \tau; \dots; \ell : \tau\}$	Record type expression

Figure 10: Record Grammar Extensions

We write $\{\ell = v; \dots\}$ to mean a record value containing at least label ℓ mapping to value v . Labels in records are unordered.

$\boxed{E \models e \Rightarrow r}$ (record extensions)

$$\begin{array}{c}
\frac{\forall i. E \models e_i \Rightarrow v_i}{E \models \{\ell_1 = e_1; \dots; \ell_n = e_n\} \Rightarrow \{\ell_1 = v_1; \dots; \ell_n = v_n\}} \text{RECORD} \qquad \frac{E \models e \Rightarrow \{\ell = v; \dots\}}{E \models e.\ell \Rightarrow v} \text{PROJECT} \\
\\
\frac{\forall i. E \models \tau_i \Rightarrow t_i}{E \models \{\ell_1 : \tau_1; \dots; \ell_n : \tau_n\} \Rightarrow \{\ell_1 : t_1; \dots; \ell_n : t_n\}} \text{RECORDTYPE}
\end{array}$$

Figure 11: Record Evaluation

Note that we could theoretically allow one to project a label from a record type, but we choose not to do it, instead saying that projection is only on record values.

It is a failure to project a missing label:

$\boxed{E \models e \Rightarrow r}$ (record extensions)

$$\frac{E \models e \Rightarrow \{\ell_1 = v_1; \dots; \ell_n = v_n\} \quad \ell \notin \{\ell_1, \dots, \ell_n\}}{E \models e.\ell \Rightarrow \text{X}} \text{PROJECTMISSINGLABEL}$$

Figure 12: Projection Failure

The natural record generator generates a record with exactly the labels declared in the type, and no more. Since there is no way to query a record for existence of a label (expressions only allow projection, above), this means that a generated record can act like any record that has extra, hidden labels. However, this *does* mean that we get incompleteness on generated functions whose domain type is the empty record: such a generated function would fail on a non-record input, but any user-defined function with such a type can never use its argument and can thus accept anything. I.e. that generated function does not simulate the behavior of a user-defined function. This incompleteness would go away if the language allows matching on sorts, like `match x with record -> e`.

$\boxed{t \uparrow v}$ (record extensions)

$$\frac{\forall i. t_i \uparrow v_i}{\{\ell_1 : t_1; \dots; \ell_n : t_n\} \uparrow \{\ell_1 = v_1; \dots; \ell_n = v_n\}} \text{GENRECORD}$$

Figure 13: Record Generator

To check records: if the candidate is not a record, or if there is some label in the record type that does not exist in the candidate value, then we have a refutation. Otherwise, we check the labels one at a time because only one incorrect label is needed for a refutation.

$\boxed{v:t \Downarrow \odot}$ (records extensions)

$$\begin{array}{c}
\frac{v \text{ is not a record}}{v:\{\ell_1:t_1;\dots;\ell_n:t_n\}\Downarrow \mathbf{X}} \text{REFUTENONRECORD} \\
\\
\frac{\{\ell_1,\dots,\ell_n\} \not\subseteq \{\ell'_1,\dots,\ell'_m\}}{\{\ell'_1=v_1;\dots;\ell'_m=v_m\}:\{\ell_1:t_1;\dots;\ell_n:t_n\}\Downarrow \mathbf{X}} \text{REFUTEMISSINGLABEL} \\
\\
\frac{\{\ell_1,\dots,\ell_n\} \subseteq \{\ell'_1,\dots,\ell'_m\} \quad \ell'_i=\ell_j \quad v_i:t_j \Downarrow \odot}{\{\ell'_1=v_1;\dots;\ell'_m=v_m\}:\{\ell_1:t_1;\dots;\ell_n:t_n\}\Downarrow \odot} \text{CHECKRECORDLABEL}
\end{array}$$

Figure 14: Record Checkers

3.2. Modules

Modules are dependent records. Dependent records are commonly used for modular programming, so we add dependent records to this language with ML-style module syntax and make them distinct from records, though they are very similar to records at their core. One way in which they differ from records is that module types cannot be evaluated eagerly because all but the first field depend on a candidate member of the type. It is most correct to eagerly evaluate this first label and put the rest in a closure, but in the implementation, we will put them all in a closure for simplicity.

values	$v ::= \dots$	
	<code>struct let $\ell = v$... let $\ell = v$ end</code>	Module value
expressions	$e ::= \dots$	
	<code>struct let $\ell = e$... let $\ell = e$ end</code>	Module expression
type values	$t ::= \dots$	
	<code>(sig val $\ell : t$ val $\ell : \tau$... val $\ell : \tau$ end, E)</code>	Module type value
types	$\tau ::= \dots$	
	<code>sig val $\ell : \tau$... val $\ell : \tau$ end</code>	Module type expression

Figure 15: Module Grammar Extensions

Note that label projection is inherited from the record grammar extensions in Figure 10.

We write `struct let  $\ell = v$  ... end` to mean a module value containing at least label ℓ mapping to value v . Labels in *module values* are unordered. Labels in *module types* are ordered.

For now, we'll assume that modules cannot contain typed `let`-statements. This will simplify the evaluation of them.

$$\boxed{E \models e \Rightarrow r} \text{ (module extensions)}$$

$$\frac{\forall i. E\{\ell_1 \mapsto v_1\}\{\dots\}\{\ell_{i-1} \mapsto v_{i-1}\} \models e_i \Rightarrow v_i}{E \models \text{struct let } \ell_1 = e_1 \dots \text{let } \ell_n = e_n \text{ end} \Rightarrow \text{struct let } \ell_1 = v_1 \dots \text{let } \ell_n = v_n \text{ end}} \text{MODULE}$$

$$\frac{E \models e \Rightarrow \text{struct let } \ell = v \dots \text{end}}{E \models e.\ell \Rightarrow v} \text{PROJECTMODULE}$$

$$\frac{E \models \tau_1 \Rightarrow t_1}{E \models \text{sig val } \ell_1 : \tau_1 \dots \text{val } \ell_n : \tau_n \text{ end} \Rightarrow \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{val } \ell_n : \tau_n \text{ end}, E \rangle} \text{MODULETYPE}$$

Figure 16: Module Evaluation

The above grammar and rules do not handle empty module types, but the implementation does.

Note that because module types contain type expressions in general instead of type values, we cannot allow the projection of a label from a module type. In contrast, we *can* allow the projection of a label from a record type, but we choose not to because of this difference.

It is a failure to project a missing label:

$$\boxed{E \models e \Rightarrow r} \text{ (module extensions)}$$

$$\frac{E \models e \Rightarrow \text{struct let } \ell_1 = v_1 \dots \text{let } \ell_n = v_n \text{ end} \quad \ell \notin \{\ell_1, \dots, \ell_n\}}{E \models e.\ell \Rightarrow \text{X}} \text{PROJECTMISSINGMODULELABEL}$$

Figure 17: Projection Failure

$$\boxed{t \uparrow v} \text{ (module extensions)}$$

$$\frac{\forall i \in \{1, \dots, n\}. t_i \uparrow v_i \quad \forall i \in \{2, \dots, n\}. E\{\ell_1 \mapsto v_1\}\{\dots\}\{\ell_{i-1} \mapsto v_{i-1}\} \models \tau_i \Rightarrow t_i}{\langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{val } \ell_n : \tau_n \text{ end}, E \rangle \uparrow \text{struct let } \ell_1 = v_1 \dots \text{let } \ell_n = v_n \text{ end}} \text{GENMODULE}$$

Figure 18: Module Generator

To check modules: if the candidate is not a record, or if there is some label in the module type that does not exist in the candidate value, then we have a refutation. Otherwise, we check the labels one at a time because only one incorrect label is needed for a refutation. This does not require that we evaluate all the types prior unless we wanted wrapping of them for usage checking while the type is evaluated.

Note that records and modules are disjoint. There is a refutation even if the candidate is a record that would pass the check when interpreted as a module.

$\boxed{v:t \Downarrow \odot}$ (module extensions)

$$\begin{array}{c}
\frac{v \text{ is not a module}}{v:\langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \mathbf{X}} \text{REFUTENONMODULE} \\
\\
\frac{v = \text{struct let } \ell'_1 = v_1 \dots \text{ let } \ell'_m = v_m \text{ end} \quad \{\ell_1, \dots, \ell_n\} \not\subseteq \{\ell'_1, \dots, \ell'_m\}}{v:\langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \mathbf{X}} \text{REFUTEMISSINGMODULELABEL} \\
\\
\frac{v = \text{struct let } \ell_1 = v_1 \dots \text{ let } \ell_n = v_n \dots \text{ end} \quad v_1 : t_1 \Downarrow \odot}{v:\langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \odot} \text{CHECKFIRSTMODULELABEL} \\
\\
\frac{v = \text{struct let } \ell_1 = v_1 \dots \text{ let } \ell_n = v_n \dots \text{ end} \quad E\{\ell_1 \mapsto v_1\}\{\dots\}\{\ell_{i-1} \mapsto v_{i-1}\} \models \tau_i \Rightarrow t_i \quad v_i : t_i \Downarrow \odot}{v:\langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \odot} \text{CHECKMODULELABEL}
\end{array}$$

Figure 19: Module Checkers

The **CHECKMODULELABEL** rule works under the assumption that values in modules are unordered, and there may be any extra labels. The associated record rule can be written this way to be consistent, but as it's not necessary, I do not rewrite it.

3.3. Variants

Variants are dual to records, and we work with them very similarly.

values $v ::= \dots$		
$\quad \mid \text{`}\ell v$		Variant value
expressions $e ::= \dots$		
$\quad \mid \text{`}\ell e$		Variant expression
$\quad \mid \text{match } e \text{ with } p \rightarrow e \mid \dots \mid p \rightarrow e \text{ end}$		Pattern match
type values $t ::= \dots$		
$\quad \mid \text{`}\ell \text{ of } t \mid \dots \mid \text{`}\ell \text{ of } t$		Variant type value
types $\tau ::= \dots$		
$\quad \mid \text{`}\ell \text{ of } \tau \mid \dots \mid \text{`}\ell \text{ of } \tau$		Variant type expression
patterns $p ::= \text{`}\ell x$		Variant pattern

Figure 20: Variant Grammar Extensions

$\boxed{E \models e \Rightarrow r}$ (variant extensions)

$$\begin{array}{c}
\frac{E \models e \Rightarrow v}{E \models \text{`}\ell e \Rightarrow \text{`}\ell v} \text{VARIANT} \\
\\
\frac{E \models e \Rightarrow v \quad v = \text{`}\ell v' \quad p_i = \text{`}\ell x \quad \nexists j < i. \text{MATCHES}(p_j, v) \quad E\{x \mapsto v'\} \models e_i \Rightarrow v_i}{E \models \text{match } e \text{ with } p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n \text{ end} \Rightarrow v_i} \text{MATCH} \\
\\
\frac{\forall i. E \models \tau_i \Rightarrow t_i}{E \models \text{`}\ell_1 \text{ of } \tau_1 \mid \dots \mid \text{`}\ell_n \text{ of } \tau_n \Rightarrow \text{`}\ell_1 \text{ of } t_1 \mid \dots \mid \text{`}\ell_n \text{ of } t_n} \text{VARIANTTYPE}
\end{array}$$

Figure 21: Variant Evaluation

We say $\text{MATCHES}(\ell x, \ell' v) = \text{true}$ when $\ell = \ell'$, and **false** otherwise. We currently do not have any patterns besides variants, so this is sufficient for now.

It is a failure to have an incomprehensive match:

$E \models e \Rightarrow r$ (variant extensions)

$$\frac{E \models e \Rightarrow v \quad \nexists i. \text{MATCHES}(p_i, v)}{E \models \text{match } e \text{ with } p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n \text{ end} \Rightarrow \mathbf{X}} \text{MATCHMISSINGPATTERN}$$

Figure 22: Pattern Match Failure

To generate a member of a variant type is a simple choice from the possible labels.

$t \uparrow v$ (variant extensions)

$$\frac{t_i \uparrow v_i}{\ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \uparrow \ell_i v_i} \text{GENVARIANT}$$

Figure 23: Variant Generator

$v : t \Downarrow \odot$ (variant extensions)

$$\frac{v \text{ is not a variant}}{v : \ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \Downarrow \mathbf{X}} \text{REFUTENONVARIANT}$$

$$\frac{\ell \notin \{\ell_1, \dots, \ell_n\}}{\ell v : \ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \Downarrow \mathbf{X}} \text{REFUTEMISSINGCONSTRUCTOR}$$

$$\frac{\ell = \ell_i \quad v : t_i \Downarrow \odot}{\ell v : \ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \Downarrow \odot} \text{CHECKVARIANT}$$

Figure 24: Variant Checkers

As the dual to records, the variant checkers have no interesting choices in them, but instead the generators do. This will be noted in the concolic variation of the system which systematically makes all possible choices.

3.4. Polymorphism

We present polymorphism before dependent functions for simplicity, though the full utility of polymorphism requires dependent functions (Section 3.8).

Here, we extend the type grammar to have **type** and parametric polymorphism.

type values $t ::=$	$\dots$	
	$\mid$ type	Type of types
	$\mid \alpha_n$	Polymorphic type
type expressions $\tau ::=$	$\dots$	
	$\mid$ type	Type of types
generated values $g ::=$	$\dots$	
	$\mid G(\alpha_n)$	Generated polymorphic value

Figure 25: Polymorphism Grammar Extensions

Generated polymorphic values must be used parametrically. Any expression that attempts to inspect a $G(\alpha_n)$ will error. There are consequently no rules for their semantics except for updates to type mismatches that cause failures. These rules are elided for brevity.

$\boxed{E \models e \Rightarrow r}$ (polymorphism extensions)

$$\frac{}{E \models \text{type} \Rightarrow \text{type}} \text{TYPE}$$

Figure 26: Evaluation Type

$\boxed{t \uparrow v}$ (polymorphism extensions)

$$\frac{n \in \mathbb{Z} \text{ fresh}}{\text{type} \uparrow \alpha_n} \text{GENTYPE} \quad \frac{}{\alpha_n \uparrow G(\alpha_n)} \text{GENPOLY}$$

Figure 27: Polymorphic Generators

This says that a generated type is a fresh polymorphic type, identified by a value n . In the concolic system, something as simple as the current time will do. Members of a polymorphic type keep around this n to remember where they came from. This allows checking.

$\boxed{v : t \Downarrow \odot}$ (polymorphism extensions)

$$\frac{v \text{ is not of the form } t}{v : \text{type} \Downarrow \mathbf{X}} \text{REFUTETYPE} \quad \frac{v \neq G(\alpha_n)}{v : \alpha_n \Downarrow \mathbf{X}} \text{REFUTEPOLY}$$

$$\frac{}{t : \text{type} \Downarrow \checkmark} \text{CONFIRMTYPE} \quad \frac{}{G(\alpha_n) : \alpha_n \Downarrow \checkmark} \text{CONFIRMPOLY}$$

Figure 28: Polymorphic Checkers

The above checker for **type** assumes that anything of the form t is a valid type. However, types could be ill formed, and we might like to run a wellformedness check by doing the following:

$$\frac{t \uparrow v \quad v : t \Downarrow \odot}{t : \text{type} \Downarrow \odot} \text{CHECKTYPE}$$

This would be expensive, but it would ensure that the type actually makes sense. It would only *not* make sense if the type has a closure in it, like with modules types, dependent function types, and recursive types, for example.

3.5. Recursion

While the programmer may write recursion with self-passing or a fixed-point combinator, such patterns hide the recursion and make it hard to finitize the program paths in future extensions. To make recursion explicit, we add **let rec** to the expression grammar and fixpoint function values $\langle \text{fix } f(x).e, E \rangle$ to the value grammar.

variables	$f, x ::=$	$(Identifiers)$	
values	$v ::=$	$\dots$	
		$ \langle \text{fix } f(x).e, E \rangle$	Fixpoint function value
expressions	$e ::=$	$\dots$	
		$ \text{let rec } f = \lambda x.e \text{ in } e$	Untyped let rec
		$ \text{let rec } f : \tau = \lambda x.e \text{ in } e$	Typed let rec

Figure 29: Recursion Grammar Extensions

$E \models e \Rightarrow r$ (recursion extensions)

$$\frac{E\{x \mapsto \langle \text{fix } f(x).e_1, E \rangle\} \models e_2 \Rightarrow r}{E \models \text{let rec } f = \lambda x.e_1 \text{ in } e_2 \Rightarrow r} \text{LETREC}$$

$$\frac{E \models e_1 \Rightarrow \langle \text{fix } f(x).e, E' \rangle \quad E \models e_2 \Rightarrow v \quad E'\{x \mapsto v\}\{f \mapsto \langle \text{fix } f(x).e, E' \rangle\} \models e \Rightarrow r}{E \models e_1 \ e_2 \Rightarrow r} \text{APPLFIX}$$

Figure 30: Let Rec Evaluation

Checkers need to be extended to accept or refute recursive functions as members of function types.

$v : t \Downarrow \odot$ (recursion extensions)

$$\frac{t_1 \Uparrow v \quad E\{x \mapsto v\}\{f \mapsto \langle \text{fix } f(x).e, E \rangle\} \models e \Rightarrow v' \quad v' : t_2 \Downarrow \odot}{\langle \text{fix } f(x).e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKREC FUN}$$

$$\frac{t_1 \Uparrow v \quad E\{x \mapsto v\}\{f \mapsto \langle \text{fix } f(x).e, E \rangle\} \models e \Rightarrow \odot}{\langle \text{fix } f(x).e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKREC FUNNONVALUE}$$

Figure 31: Check Recursive Functions

Next are the standard rules for checking typed recursive **let**-expressions. Like the non-recursive **let**-expressions, there is a choice as to whether to check the definition or run the body.

$E \models e \Rightarrow r$ (recursion extensions)

$$\frac{E \models \tau \Rightarrow t \quad E\{f \mapsto \langle \text{fix } f(x).e, E \rangle\} \models e' \Rightarrow r}{E \models \text{let rec } f : \tau = \lambda x.e \text{ in } e' \Rightarrow r} \text{LETRECTYPED}$$

$$\frac{E \models \tau \Rightarrow t \quad \langle \text{fix } f(x).e, E \rangle : t \Downarrow \odot}{E \models \text{let rec } f : \tau = \lambda x.e \text{ in } e' \Rightarrow \odot} \text{LETREC CHECK}$$

Figure 32: Typed Recursive Function Evaluation

3.6. Recursive types

Since the evaluation semantics are eager, recursive types cannot be encoded; rather, they need special support. Recursive types are represented as closures that unroll when they are used to check and generate values.

type values	$t ::= \dots$	
	$ \ \langle \mu x. \tau, E \rangle$	Recursive type value
type expressions	$\tau ::= \dots$	
	$ \ \mu x. \tau$	Recursive type expression

Figure 33: Recursive Types Grammar Extensions

$E \models e \Rightarrow r$

 (recursive types extensions)

$$\frac{}{E \models \mu x. \tau \Rightarrow \langle \mu x. \tau, E \rangle} \text{Mu}$$

Figure 34: Evaluate Recursive Types

Note that because recursive type values have a closure, they are not well formed by assumption. We may like to do a wellformedness check on them. Such a wellformedness check would be that no generated member can be refuted to have the type that generated it. We do no such checking right now, and we assume that all types written are well formed.

As said above, recursive types unroll when used to generate and check:

$t \uparrow v$

 (recursive types extensions)

$$\frac{E\{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow t \quad t \uparrow v}{\langle \mu x. \tau, E \rangle \uparrow v} \text{GENMu}$$

Figure 35: Generate Recursive Types

$v : t \Downarrow \odot$

 (recursive types extensions)

$$\frac{E\{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow t \quad v : t \Downarrow \odot}{v : \langle \mu x. \tau, E \rangle \Downarrow \odot} \text{CHECKMu}$$

$$\frac{E\{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow \odot}{v : \langle \mu x. \tau, E \rangle \Downarrow \odot} \text{CHECKMuNonValue}$$

Figure 36: Check Recursive Types

All the work in the checker is done by the fixed point semantics of recursive type evaluation. The type is unrolled one layer, and then the candidate is checked. If the candidate needs a recursive checking, then the type is unrolled again on demand.

We need **CHECKMuNonValue** in case the type is not well formed or it has a checker inside of it; it may vanish and still be well formed.

With the above rules, we expect the following program to diverge. We may like to handle it, eventually.

```

let t = mu x.
  let _ : x = 0 in
  int
let _ : t = 10

```

3.7. Refinement types

Refinement types are easy in semantic typing where we are used to running code in the midst of type checking. We only need to verify the predicate holds on checked and generated values, and therefore generators will now be allowed to vanish. Hence the generator relation is now $t \uparrow r$, but this is not updated everywhere, and we are a little sloppy and still consistently write it $t \uparrow v$.

The predicate is an expression that depends on a candidate member, so it is a closure. The base type, on the other hand, can be evaluated eagerly.

```

type values  $t ::= \dots$ 
              |  $\{x : t \mid \langle e, E \rangle\}$  refinement type value
types  $\tau ::= \dots$ 
              |  $\{x : \tau \mid e\}$  refinement type expression

```

Figure 37: Refinement Type Grammar Extensions

$E \models e \Rightarrow r$ (refinement extensions)

$$\frac{E \models \tau \Rightarrow t}{E \models \{x : \tau \mid e\} \Rightarrow \{x : t \mid \langle e, E \rangle\}} \text{REFINEMENTTYPE}$$

Figure 38: Refinement Type Evaluation

To support generating refinement types, we must be prepared for the predicate to fail to evaluate or to evaluate to **false**. This means that generators can fail to produce a value, which was already the case (but glossed over) with modules. This means the relation form is $t \uparrow r$, but we do not show this everywhere.

$t \uparrow v$ (refinement extensions)

$$\frac{t \uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow \text{true}}{\{x : t \mid \langle e, E \rangle\} \uparrow v} \text{GENREFINEMENT}$$

$$\frac{t \uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow \text{false}}{\{x : t \mid \langle e, E \rangle\} \uparrow \checkmark} \text{GENREFINEMENTVANISH}$$

$$\frac{t \uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow \odot}{\{x : t \mid \langle e, E \rangle\} \uparrow \odot} \text{GENREFINEMENTNONEVAL}$$

$$\frac{t \uparrow \odot}{\{x : t \mid \langle e, E \rangle\} \uparrow \odot} \text{GENREFINEMENTNONVALUE}$$

Figure 39: Refinement Type Generator

Now that generators can fail to produce a value, we should technically update all generators to handle the case that a component type does not generate a value, like in **GENREFINEMENTNONVALUE**. For brevity, these rules are currently not shown in this document.

To refute a refinement type, we can either refute the base type, show the refinement fails to evaluate, or show it evaluates to **false**. When given the choice, it should be preferable to first try checking the base type because the predicate does not mean much if the base type is wrong.

$\boxed{v:t \Downarrow \odot}$ (refinement extensions)

$$\begin{array}{c}
\frac{v:t \Downarrow \odot}{v:\{x:t \mid \langle e, E \rangle\} \Downarrow \odot} \text{CHECKREFINEMENTTYPE} \\
\\
\frac{E\{x \mapsto v\} \models e \Rightarrow \text{false}}{v:\{x:t \mid \langle e, E \rangle\} \Downarrow \times} \text{REFUTEREFINEMENTPREDICATE} \\
\\
\frac{E\{x \mapsto v\} \models e \Rightarrow \text{true}}{v:\{x:t \mid \langle e, E \rangle\} \Downarrow \checkmark} \text{CONFIRMREFINEMENTPREDICATE} \\
\\
\frac{E\{x \mapsto v\} \models e \Rightarrow \odot}{v:\{x:t \mid \langle e, E \rangle\} \Downarrow \odot} \text{TRYREFINEMENTPREDICATE}
\end{array}$$

Figure 40: Refinement Type Checkers

For the purposes of finding refutations, there is no good reason to build proof trees using the `GENREFINEMENTVANISH` rule. It is wasted effort to generate many values that do not pass the refinement. In the concolic ruleset, where the proof trees are systematically explored, this observation is exploited to avoid searching parts of the path tree.

3.8. Dependent functions

Dependent functions are semantically easy, but they add many cases for checking because they need to be compatible with regular functions in all cases and all directions. We first extend the grammar with dependent function types and generated dependent functions.

$$\begin{array}{ll}
\text{type values } t ::= \dots & \\
\quad \mid (x:t) \rightarrow \langle \tau, E \rangle & \text{Dependent function type value} \\
\text{types } \tau ::= \dots & \\
\quad \mid (x:\tau) \rightarrow \tau & \text{Dependent function type} \\
\text{generated values } t ::= \dots & \\
\quad \mid G((x:t) \rightarrow \langle \tau, E \rangle) & \text{Generated dependent function}
\end{array}$$

Figure 41: Dependent Functions Grammar Extensions

$\boxed{E \models e \Rightarrow r}$ (dependent function extensions)

$$\frac{E \models \tau_1 \Rightarrow t}{E \models (x:\tau_1) \rightarrow \tau_2 \Rightarrow (x:t) \rightarrow \langle \tau_2, E \rangle} \text{DEPFUN}$$

Figure 42: Evaluate Dependent Function Types

$\boxed{t \Uparrow v}$ (dependent function extensions)

$$\frac{}{(x:t) \rightarrow \langle \tau, E \rangle \Uparrow G((x:t) \rightarrow \langle \tau, E \rangle)} \text{GENDEPFUN}$$

Figure 43: Generate Dependent Functions

The evaluation and generation rules above are trivial. It is the checkers that raise many cases because dependent functions must be compatible with regular functions, in both directions.

We present the rules in several figures to break up the cases. First, we check function values against dependent function types, which are an extension to the original rules.

$v : t \Downarrow \odot$

 (dependent function extensions and overwrites)
$$\frac{t \Uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow v' \quad E'\{x' \mapsto v\} \models \tau \Rightarrow t' \quad v' : t' \Downarrow \odot}{\langle \lambda x.e, E \rangle : (x' : t) \rightarrow \langle \tau, E' \rangle \Downarrow \odot} \text{CHECKDEPFUN}$$

$$\frac{t \Uparrow v \quad E\{x \mapsto v\} \{f \mapsto \langle \text{fix } f(x).e, E \rangle\} \models e \Rightarrow v' \quad E'\{x' \mapsto v\} \models \tau \Rightarrow t' \quad v' : t' \Downarrow \odot}{\langle \text{fix } f(x).e, E \rangle : (x' : t) \rightarrow \langle \tau, E' \rangle \Downarrow \odot} \text{CHECKDEPRECFUN}$$

Figure 44: Check Dependent Functions

Next, we overwrite the old `CHECKGENFUNLEFT` rule to handle dependent function types, too. Since the codomains are untouched, one rule handles all cases.

$v : t \Downarrow \odot$

 (dependent function overwrites)
$$\frac{t' \in \{t'_1 \rightarrow t'_2, (x : t'_1) \rightarrow \langle \tau', E' \rangle\} \quad t \in \{t_1 \rightarrow t_2, (x : t_1) \rightarrow \langle \tau, E \rangle\} \quad t_1 \Uparrow v \quad v : t'_1 \Downarrow \odot}{G(t') : t \Downarrow \odot} \text{CHECKGENFUNLEFT}$$

Figure 45: Check Dependent Functions (Left)

And finally are the rules to handle all combinations of codomains not already handled in the base system. In every case here, some codomain depends on a member of the domain, so we begin by generating a value from the checked domain. This is strictly more expensive than with non-dependent functions, so for performance reasons, we do not make all functions dependent by default, and dependent functions should be reserved for only the cases they are absolutely needed.

$v : t \Downarrow \odot$

 (dependent function extensions)
$$\frac{t_1 \Uparrow v_1 \quad E'\{x' \mapsto v_1\} \models \tau' \Rightarrow t'_2 \quad E\{x \mapsto v_1\} \models \tau \Rightarrow t_2 \quad t'_2 \Uparrow v_2 \quad v_2 : t_2 \Downarrow \odot}{G((x' : t'_1) \rightarrow \langle \tau', E' \rangle) : (x : t_1) \rightarrow \langle \tau, E \rangle \Downarrow \odot} \text{CHECKDEP2FUNRIGHT}$$

$$\frac{t_1 \Uparrow v_1 \quad E\{x \mapsto v_1\} \models \tau \Rightarrow t_2 \quad t'_2 \Uparrow v_2 \quad v_2 : t_2 \Downarrow \odot}{G(t'_1 \rightarrow t'_2) : (x : t_1) \rightarrow \langle \tau, E \rangle \Downarrow \odot} \text{CHECKDEPFUNRIGHT}$$

$$\frac{t_1 \Uparrow v_1 \quad E'\{x' \mapsto v_1\} \models \tau' \Rightarrow t'_2 \quad t'_2 \Uparrow v_2 \quad v_2 : t_2 \Downarrow \odot}{G((x' : t'_1) \rightarrow \langle \tau', E' \rangle) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKGENFUNRIGHT}$$

Figure 46: Check Dependent Functions (Right)

Notice how we cover these four rules in one by overwriting the base system's `CHECKGENFUNLEFT` rule:

$$\frac{t_1 \Uparrow v \quad v : t'_1 \Downarrow \odot}{G((x' : t'_1) \rightarrow \langle \tau', E' \rangle) : (x : t_1) \rightarrow \langle \tau, E \rangle \Downarrow \odot}$$

$$\begin{array}{c}
\frac{t_1 \uparrow v \quad v : t'_1 \Downarrow \odot}{G((x' : t'_1) \rightarrow \langle \tau', E' \rangle) : t_1 \rightarrow t_2 \Downarrow \odot} \\
\frac{t_1 \uparrow v \quad v : t'_1 \Downarrow \odot}{G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \\
\frac{t_1 \uparrow v \quad v : t'_1 \Downarrow \odot}{G((x' : t'_1) \rightarrow \langle \tau', E' \rangle) : t_1 \rightarrow t_2 \Downarrow \odot}
\end{array}$$

We now see how generated dependent functions are used. They are just like regular generated functions, but the codomain type must be evaluated in case a value is returned from the application.

$E \models e \Rightarrow r$ (dependent function extensions)

$$\begin{array}{c}
\frac{E \models e_1 \Rightarrow G((x : t) \rightarrow \langle \tau, E' \rangle) \quad E \models e_2 \Rightarrow v \quad E' \{x \mapsto v\} \models \tau \Rightarrow t' \quad t' \uparrow v'}{E \models e_1 \ e_2 \Rightarrow v'} \text{APPLDEPGENFUN} \\
\\
\frac{E \models e_1 \Rightarrow G((x : t) \rightarrow \langle \tau, E' \rangle) \quad E \models e_2 \Rightarrow v \quad v : t \Downarrow \odot}{E \models e_1 \ e_2 \Rightarrow \odot} \text{APPLDEPCHECK}
\end{array}$$

Figure 47: Evaluate Generated Dependent Functions

Like with regular functions, checkers for dependent functions might not evaluate.

$v : t \Downarrow \odot$ (dependent function extensions)

$$\begin{array}{c}
\frac{t \uparrow v \quad E \{x \mapsto v\} \models e \Rightarrow \odot}{\langle \lambda x. e, E \rangle : (x' : t) \rightarrow \langle \tau, E' \rangle \Downarrow \odot} \text{CHECKDEPFUNNONVALUE} \\
\\
\frac{t \uparrow v \quad E \{x \mapsto v\} \{f \mapsto \langle \text{fix } f(x). e, E \rangle\} \models e \Rightarrow \odot}{\langle \text{fix } f(x). e, E \rangle : (x' : t) \rightarrow \langle \tau, E' \rangle \Downarrow \odot} \text{CHECKDEPRECFUNNONVALUE}
\end{array}$$

Figure 48: Dependent Function Checker via Non-evaluation

There are also rules for non-evaluation of the type itself, like this:

$$\frac{t \uparrow v \quad E \{x \mapsto v\} \models e \Rightarrow v' \quad E' \{x' \mapsto v'\} \models \tau \Rightarrow \odot}{\langle \lambda x. e, E \rangle : (x' : t) \rightarrow \langle \tau, E' \rangle \Downarrow \odot}$$

But these are wordy, so I do not write any more of them. It is easy enough to see that the errors must propagate. The rules I write above are focused only on the meaning of the types.

One can now see how polymorphic functions are written. They are dependently typed functions that take a type as an argument. For example, a polymorphic **apply** function:

```

let f : (a : type) -> (a -> b) -> a -> b =
  fun a f x -> f x

let _ = f int (fun i -> i + 1) 0

```

3.9. Singleton types

Singleton types are helpful for defining type equalities in signatures. To include singleton types, we can use intensional equality (incomplete) or restrict singleton types to only work on types, in which case we check subsets in both directions.

We take the latter approach because it is complete, and we call it “singleton”.

type values $t ::= \dots$
 | $\text{SINGLETYPE}(t)$ Singleton type value
 type expressions $\tau ::= \dots$
 | $\text{SINGLETYPE}(\tau)$ Singleton type expression

Figure 49: Singleton Grammar Extensions

$E \models e \Rightarrow r$ (singleton extensions)

$$\frac{E \models \tau \Rightarrow t}{E \models \text{SINGLETYPE}(\tau) \Rightarrow \text{SINGLETYPE}(t)} \text{SINGLETYPE}$$

Figure 50: Evaluate Singleton

$t \uparrow v$ (singleton extensions)

$$\frac{}{\text{SINGLETYPE}(t) \uparrow t} \text{GENSINGLETYPE}$$

Figure 51: Generate Singleton

To check if $t' \in \llbracket \text{SINGLETYPE}(t) \rrbracket$ is to check that $\llbracket t' \rrbracket = \llbracket t \rrbracket$. We can get a refutation if $\llbracket t' \rrbracket \not\subseteq \llbracket t \rrbracket$ or $\llbracket t' \rrbracket \not\supseteq \llbracket t \rrbracket$. We have a rule to check each.

$v : t \Downarrow \odot$ (singleton extensions)

$$\frac{t' \uparrow v \quad v : t \Downarrow \odot}{t' : \text{SINGLETYPE}(t) \Downarrow \odot} \text{CHECKSINGLETYPELEFT} \qquad \frac{t \uparrow v \quad v : t' \Downarrow \odot}{t' : \text{SINGLETYPE}(t) \Downarrow \odot} \text{CHECKSINGLE typerIGHT}$$

Figure 52: Check Singleton

A typical usage of `singleton` is something like this:

```

let Make_list : (T : sig val t : type end) -> sig
  val t : singleton (list T.t)
  val empty : t
end = fun T -> struct
  let t = list T.t
  let empty = []
end

```

3.10. Unit

Beginning with `unit`, the remainder of the language extensions are easy. `unit` is a type with one member, where that member is useless.

The unit value `()` is useless, but it may be differentiated from other values by matching (which is not expressed in the grammar or rules below). Thus, it is not incomplete; if there were no way to use the value, then there is no way to differentiate it from anything else, and thus no user-defined function with type `unit → unit` could fail on a non-`unit` argument.

$$\begin{array}{lcl}
v & ::= & \dots \\
& | & () \quad \text{Unit value} \\
\text{type values } t & ::= & \dots \\
& | & \text{unit} \quad \text{Unit type value}
\end{array}$$

Figure 53: Unit Grammar Extensions

$E \models e \Rightarrow r$ (unit extensions)

$$\frac{}{E \models () \Rightarrow ()} \text{UNIT} \qquad \frac{}{E \models \text{unit} \Rightarrow \text{unit}} \text{UNITTYPE}$$

Figure 54: Evaluate Unit

$t \uparrow v$ (unit extensions)

$$\frac{}{\text{unit} \uparrow ()} \text{GENUNIT}$$

Figure 55: Generate Unit

$v : t \Downarrow \odot$ (unit extensions)

$$\frac{}{() : \text{unit} \Downarrow \checkmark} \text{CONFIRMUNIT} \qquad \frac{v \neq ()}{v : \text{unit} \Downarrow \checkmark} \text{REFUTEUNIT}$$

Figure 56: Check Unit

3.11. Bottom

We use $\llbracket \perp \rrbracket = \emptyset$ to guide the rules for the bottom type.

$$\begin{array}{lcl}
\text{type values } t & ::= & \dots \\
& | & \perp \quad \text{Bottom type value}
\end{array}$$

Figure 57: Bottom Grammar Extensions

$E \models e \Rightarrow r$ (bottom extensions)

$$\frac{}{E \models \perp \Rightarrow \perp} \text{BOTTOMTYPE}$$

Figure 58: Evaluate Bottom

There is no member of the bottom type, so generation vanishes.

$t \uparrow v$ (bottom extensions)

$$\frac{}{\perp \uparrow \checkmark} \text{GENBOTTOM}$$

Figure 59: Generate Bottom

Any check with bottom is a refutation because it has no members.

$\boxed{v:t \Downarrow \odot}$ (bottom extensions)

$$\frac{}{v: \perp \Downarrow \times} \text{CHECKBOTTOM}$$

Figure 60: Check Bottom

3.12. Lists

Lists are a special case of recursive functions and variants. They can be handled specially for efficiency, or they can be desugared to variants.

A desugaring to variants would be something like `list = λx.μx'.`ℓempty of unit | `ℓcons of x * x'`. A native representation would duplicate a lot of logic from variants and recursive types (and records or tuples). The implementation does a native representation.

4. Type splaying

The goal of type splaying is to eliminate recursion and present a terminating variation of the system. The following set of rules describe this variation, and it supposes all recursion is annotated with types.

The type splayed system is incomplete, but we conjecture it is sound. That is, there are “false refutations”—the type refutations are not guaranteed to actually be counterexamples of type membership. On the other hand, if there is a type error in the program, then type splaying will find it. Since there are finitely many type splayed proof trees, it is practical to enumerate all of them and show the lack of a refutation. In such a case, the program is proven well typed.

4.1. Splaying Recursive Functions

We use a fixpoint typing rule to splay recursive functions. When checking, instead of checking a recursive function, we close the recursion with a generated member of the annotated type. We do the same when evaluating the body so as to still eliminate recursion in usages of the function, not just when checking.

$\boxed{E \models e \Rightarrow r}$ (type splaying rules)

$$\frac{E \models \tau \Rightarrow t \quad t \Uparrow v \quad \langle \lambda x.e, E\{f \mapsto v\} \rangle : t \Downarrow \odot}{E \models \text{let rec } f : \tau = \lambda x.e \text{ in } e' \Rightarrow \odot} \text{LETRECCHCKSPAYED}$$

$$\frac{E \models \tau \Rightarrow t \quad t \Uparrow v \quad E\{f \mapsto \langle \lambda x.e, E\{f \mapsto v\} \rangle\} \models e' \Rightarrow r}{E \models \text{let rec } f : \tau = \lambda x.e \text{ in } e' \Rightarrow r} \text{LETRECEVALSPAYED}$$

Figure 61: Type Splaying Evaluation

This means that type splayed programs have limited ability to explore data. For example, a `reverse` function on a list only explores the head of the list, and it generates a random tail for the recursive call.

4.2. Splaying Recursive Types

The goal of type splaying is to eliminate recursion. Recursive types introduce potentially-unbounded recursion when generating, which we would like to remove. Recall how when splaying recursive functions, the actual function body only evaluates one level deep. If we did the same with recursive types, then members would only be usable to one level, which is overly

restrictive. Instead, we can notice that a type splayed program has limited ability to use values, supposing all recursion is type-annotated. If we generate recursive members only as large as the program can possibly use them, then we have still eliminated recursion but have not been overly cautious.

A naive approach is to allow recursive types to unroll up to some static limit. This is naive because it may generate significantly more members than get used. There may be an astounding number of members of a recursive type within a small unrolling, but very few ever get used meaningfully by the program. Instead, values should be generated on demand!

To generate on demand requires laziness, so in this section we extend evaluation to keep a lazy state.

The grammar is extended with lazy values and lazy state.

lazy identifiers	$z ::=$	$(Identifiers)$	
values	$v ::=$	$\dots$	
		$ z$	
lazy values	$l ::=$	$L(\langle \mu x. \tau, E \rangle)$	Lazily generated value
		$ v$	Regular value
lazy states	$\zeta ::=$	$\{z \mapsto l, \dots, z \mapsto l\}$	
evaluation result	$r ::=$	v, ζ	
		$ \odot$	

Figure 62: Lazy Grammar Extensions

Evaluation rules will propagate the state. This is a trivial extension and is assumed to be clear, so we do not rewrite all of the unrelated rules to do so. Evaluation becomes $\zeta, E \models e \Rightarrow r$ to mean “with initial lazy state ζ and in environment E , the expression e evaluates to result r ”. Similarly, generators accept and produce a lazy state, but checkers do not produce a final lazy state because it would never be used; checkers are always the end of an evaluation, and hence $\odot$ does not come with a final lazy state.

The new rules follow.

$\zeta \models t \uparrow v, \zeta$ (lazy extensions)

$$\frac{z \text{ fresh}}{\zeta \models \langle \mu x. \tau, E \rangle \uparrow z, \zeta \{z \mapsto L(\langle \mu x. \tau, E \rangle)\}} \text{GENMuLAZY}$$

Figure 63: Generate Lazy

The above rule is a bit sloppy (like **type** generation) and assumes we can generate a fresh identifier. The sloppiness is resolved when the system is extended to concolic evaluation, so we will ignore the issue for now.

Since we now have two forms of values: values in weak head normal form, and values that are lazy, we add a “force evaluation” that evaluates a value to weak head normal form when the shape of the value is needed. We present it here and use it in the remaining rules.

$\boxed{\zeta \models v \Rightarrow r}$ (lazy extensions)

$$\begin{array}{c}
\frac{(z \mapsto v) \in \zeta'}{\zeta \models z \Rightarrow v, \zeta'} \text{FORCELOOKUP} \quad \frac{v \text{ is not of the form } z}{\zeta \models v \Rightarrow v, \zeta} \text{FORCEVALUE} \\
\\
\frac{\begin{array}{c} (z \mapsto L(\langle \mu x. \tau, E \rangle)) \in \zeta_0 \\ \zeta_0, E\{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow t, \zeta_1 \quad \zeta_1 \models t \uparrow v, \zeta_2 \quad \zeta_2 \models v \Rightarrow v', \zeta_3 \end{array}}{\zeta_0 \models z \Rightarrow v', \zeta_3\{z \mapsto v'\}} \text{FORCEGEN}
\end{array}$$

Figure 64: Force Values

Note that the actual logic of generating a member of a recursive type is in the **FORCEGEN** rule because the **GENMULAZY** rule only puts off the generation until later.

Also note that forced evaluation only works on values. We now extend the notation to evaluate expressions under an environment, too:

$\boxed{\zeta, E \models e \Rightarrow r}$ (lazy extensions)

$$\frac{\zeta_0, E \models e \Rightarrow v, \zeta_1 \quad \zeta_1, E \models v \Rightarrow v', \zeta_2}{\zeta_0, E \models e \Rightarrow v', \zeta_2} \text{FORCEEXPR}$$

Figure 65: Force Evaluate Expressions

Now, whenever the shape of a value is needed, it must use forced evaluation. To demonstrate, here are the rules for function application and type checking. The rest of the rules are left implicit.

$\boxed{\zeta, E \models e \Rightarrow r}$ (lazy updates)

$$\begin{array}{c}
\frac{\zeta_0, E \models e_1 \Rightarrow \langle \lambda x. e, E' \rangle, \zeta_1 \quad \zeta_1, E \models e_2 \Rightarrow v, \zeta_2 \quad \zeta_2, E'\{x \mapsto v\} \models e \Rightarrow r}{\zeta_0, E \models e_1 \ e_2 \Rightarrow r} \text{APPL} \\
\\
\frac{\zeta_0, E \models e_1 \Rightarrow v, \zeta_1 \quad \zeta_1, E \models \tau \Rightarrow t, \zeta_2 \quad \zeta_2 \models v : t \Downarrow \odot}{\zeta_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \odot} \text{LETCHECK}
\end{array}$$

Figure 66: Lazy Evaluation

Similarly, checkers will now tend to force the values they are checking. The special case is when checking with recursive types, where we want to specially handle lazy values so as to avoid recursion. First, we show the rules for recursive type checkers.

$\boxed{\zeta \models v : t \Downarrow \odot}$ (lazy extensions)

$$\begin{array}{c}
\frac{(z \mapsto v) \in \zeta \quad \zeta \models v : \langle \mu x. \tau, E \rangle \Downarrow \odot}{\zeta \models z : \langle \mu x. \tau, E \rangle \Downarrow \odot} \text{CHECKMuVAL} \\
\\
\frac{\begin{array}{c} (z \mapsto L(\langle \mu x'. \tau', E' \rangle)) \in \zeta \quad \zeta_0 \models \text{type} \uparrow v, \zeta_1 \quad \zeta_1, E' \{x' \mapsto v\} \models \tau' \rightarrow t', \zeta_2 \\ \zeta_2, E \{x \mapsto v\} \models \tau \rightarrow t, \zeta_3 \quad \zeta_3 \models t' \uparrow v', \zeta_4 \quad \zeta_4 \models v' : t \Downarrow \odot \end{array}}{\zeta_0 \models z : \langle \mu x. \tau, E \rangle \Downarrow \odot} \text{CHECKMuLAZY}
\end{array}$$

Figure 67: Lazy Checkers

The idea is that when a lazy value is checked against a recursive type, we plug the recursive points with an fresh, unusable type. This is strictly incomplete: if the recursive points do not line up, then the check goes wrong, even if semantically there is no issue.

We further see that if a lazy value is never used or checked, then it is never generated. If that value could not have existed (e.g. the type is empty), then this system may find errors that were not possible. Again, this is incomplete but still sound.

For now, we elide the rules that handle z like in **CHECKMuVAL** but for other types. They are possibly best represented with a “forced check” relation, similar to forced evaluation.

5. Wrap

5.1. Base wrap

In this section we present wrapping on only the base system with integers, booleans, and functions.

Wrapping is our way of doing usage checking. If a value has been type annotated, then we want to check that usages of the value conform to the type annotation, not just that the value is in the annotated type; specifically, function types must be used on well-typed arguments. This is simply done by “wrapping” the value with the type so that wherever it goes, it takes the type with it. Then, when the value is used, we do some type checking to make sure it is used properly.

First, let’s look at an example of a refutation that arises from wrap but not from the base system.

```
let f : int -> int = fun x -> x
let _ = f true
```

While the above program is semantically well typed, the programmer might prefer that the type checker refutes it because `f` is used with an argument outside the annotated domain.

First we extend the grammar to include wrapped values. Notice that wrapped values are always functions, but to reduce grammar cases, we do not spell out every value that acts like a function, nor write grammar terminals to organize them as such (e.g. we do not list them like $w(\langle \lambda x. e, E \rangle, t \rightarrow t)$, etc.). It’s done this way for simplicity and conciseness.

$$\begin{array}{ll}
\text{values } v ::= & \dots \\
& | \quad w \quad \text{Wrapped values} \\
\text{wrapped values } w ::= & w(v, t \rightarrow t) \quad \text{Wrapped function}
\end{array}$$

Figure 68: Wrap Grammar Extensions

We begin with how wrapped values are introduced: at type annotations. We write $v : t \hookrightarrow v_w$ to mean “value v wrapped with type t is v_w ”. This is how it is remembered that v has type t —we make a new value v_w for it.

$$\boxed{E \models e \Rightarrow r} \text{ (overwrite LETTYPED)}$$

$$\frac{E \models e_1 \Rightarrow v \quad E \models \tau \Rightarrow t \quad v : t \hookrightarrow v_w \quad E\{x \mapsto v_w\} \models e_2 \Rightarrow r}{E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow r} \text{LETTYPED}$$

Figure 69: Wrapped type annotations

Since there is still the **LET**CHECK rule, unchanged, we know that wherever there is a wrapped value $W(v, t)$, there has already been a chance to refute the typing. Therefore, we can work with the assumption that $v \in \llbracket t \rrbracket$ and use this to our advantage for efficient checking and usage.

Now, here are some simple wrappers:

$$\boxed{v : t \hookrightarrow v}$$

$$\frac{}{n : \text{int} \hookrightarrow n} \text{WRAPINT} \qquad \frac{}{b : \text{bool} \hookrightarrow b} \text{WRAPBOOL}$$

$$\frac{v \text{ is not of the form } w \quad t = t_1 \rightarrow t_2}{v : t \hookrightarrow W(v, t)} \text{WRAPFUN} \qquad \frac{t = t_1 \rightarrow t_2}{W(v, t') : t \hookrightarrow W(v, t)} \text{REWRAPFUN}$$

Figure 70: Wrappers

Integers and booleans already contain all the type information they need, so wrapping is a no-op. We wrap functions by creating wrapped values, and we re-wrap functions by discarding the old wrapping and adding the new one. This works because the new wrapping type t has been checked, and if it was not a supertype of the old wrapping type t' , then the check would have failed. Therefore, $\llbracket t \rrbracket \supseteq \llbracket t' \rrbracket$, and it is the more restrictive wrapping—there is no point in keeping the old one because the new one will catch all usage errors itself.

Notice that there are no rules for wrapping the wrong thing. For example, we have no way to wrap an integer with a function type. Since the type annotation has been checked, we can assume such a case will not happen, or else that check can yield a refutation. In the implementation, instead of erroring, we simply make all “wrong” wrappings a no-op.

Now we show how wrappers check usages to ensure that there is no undefined behavior. Since we only type-check functions (at annotations) on well-typed inputs, any ill-typed input may induce undefined behavior; we will want wrappers to turn this undefined behavior into an error and cause evaluation to fail in case the check goes wrong:

$$\boxed{E \models e \Rightarrow r} \text{ (wrap extensions)}$$

$$\frac{E \models e_1 \Rightarrow W(v, t_1 \rightarrow t_2) \quad E \models e_2 \Rightarrow v' \quad v' : t_1 \Downarrow \odot}{E \models e_1 \ e_2 \Rightarrow \odot} \text{APPLWRAPCHECK}$$

Figure 71: Check Wrapped Application

Let’s pause here to comment on this rule. When v is $G(t'_1 \rightarrow t'_2)$, we do not need to do $v' : t'_1 \Downarrow \odot$. This may seem counterintuitive at first because generated functions usually check their

arguments. However, it is not necessary when the generated function is wrapped with another type. This is because we know that v has already been type checked against $t_1 \rightarrow t_2$, so whenever $t_1 \uparrow v_1$, we must have $v_1 : t'_1 \Downarrow \checkmark$. Therefore, $\llbracket t_1 \rrbracket \subseteq \llbracket t'_1 \rrbracket$, so if $v' : t'_1 \Downarrow \mathbf{X}$, then $v' : t_1 \Downarrow \mathbf{X}$. i.e. **APPLWRAPCHECK** derives a rule that has $v' : t'_1 \Downarrow \mathbf{X}$. Since refutations are all we really care about, and confirmations are only there to help us find those refutations, this is evidence enough that we don't need to check against t'_1 .

Outside of checking usages, wrapped values also need to be used normally and propagate wrappers.

$E \models e \Rightarrow r$ (wrap extensions)

$$\frac{E \models e_1 \Rightarrow W(\langle \lambda x.e, E' \rangle, t_1 \rightarrow t_2) \quad E \models e_2 \Rightarrow v \quad E' \{x \mapsto v\} \models e \Rightarrow v' \quad v' : t_2 \hookrightarrow v'_w}{E \models e_1 \ e_2 \Rightarrow v'_w} \text{APPLWRAPFUN}$$

$$\frac{E \models e_1 \Rightarrow W(G(t'_1 \rightarrow t'_2), t_1 \rightarrow t_2) \quad E \models e_2 \Rightarrow v \quad t'_2 \uparrow v' \quad v' : t_2 \hookrightarrow v'_w}{E \models e_1 \ e_2 \Rightarrow v'_w} \text{APPLWRAPGENFUN}$$

Figure 72: Evaluate wrapped applications

To propagate the wrapping, the result of an application to a wrapped function is wrapped. On the other hand, since wrapped values are already type checked, and the type checking will find any misuse of a function's argument, we know that wrapped values do not misuse their arguments. Therefore, the arguments to wrapped functions are not wrapped because it would be pointless; only results are wrapped. For example, v in **APPLWRAPFUN** need not be wrapped.

Wrapped values have special checking rules. Considering the discussion above, there are some assumptions about subtyping we may make, and therefore some checks we may skip. We comment further after the rules.

$v : t \Downarrow \odot$ (wrap extensions)

$$\frac{t_1 \uparrow v' \quad v' : t'_1 \Downarrow \odot}{W(v, t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKWRAPANYFUNLEFT}$$

$$\frac{t_1 \uparrow v \quad E \{x \mapsto v\} \models e \Rightarrow v' \quad v' : t'_2 \hookrightarrow v'_w \quad v'_w : t_2 \Downarrow \odot}{W(\langle \lambda x.e, E \rangle, t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKWRAPFUNRIGHT}$$

$$\frac{t'_2 \uparrow v \quad v : t'_2 \hookrightarrow v_w \quad v_w : t_2 \Downarrow \odot}{W(G(t'_1 \rightarrow t'_2), t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKWRAPGENFUNRIGHT}$$

Figure 73: Check wrapped values

By the same argument we made for **APPLWRAPCHECK** about subtyping, we do not need this rule:

$$\frac{t_1 \uparrow v' \quad v' : t'_1 \Downarrow \odot}{W(G(t'_1 \rightarrow t'_2), t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot}$$

We already can assume $\llbracket t'_1 \rrbracket \subseteq \llbracket t_1 \rrbracket$, so if $v' : t'_1 \Downarrow \mathbf{X}$, then $v' : t_1 \Downarrow \mathbf{X}$, i.e. **CHECKWRAPANYFUNLEFT** is enough.

We still need to propagate non-evaluation through the wrapped checkers, too:

$\boxed{v : t \Downarrow \odot}$ (wrap extensions)

$$\frac{t_1 \Uparrow v \quad E\{x \mapsto v\} \models e \Rightarrow \odot}{w(\langle \lambda x. e, E \rangle, t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKWRAPFUNNONVALUE}$$

Figure 74: Wrapped checker via non-evaluation

Next, we present the wrappers for various extensions.

5.2. Record wrapping

There is a design decision to make with respect to wrapping records: can a wrapped record contain more than the declared labels?

We want the behavior to be consistent with how an encoded record would be wrapped, where a record is encoded by a function from a variant label to a value for that label. If records were encoded this way, then the projection of an undeclared label is a usage error; it would be applying a function with a label not in the declared type. For consistency and to preserve the duality of records and variants, we decide that a record wrapper removes extra fields so that projection of an undeclared label is an error.

$\boxed{v : t \hookrightarrow v}$ (record extensions)

$$\frac{\forall i. v_i : t_i \hookrightarrow v_i^w}{\{\ell_1 = v_1; \dots; \ell_n = v_n; \dots\} : \{\ell_1 : t_1; \dots; \ell_n : t_n\} \hookrightarrow \{\ell_1 = v_1^w; \dots; \ell_n = v_n^w\}} \text{WRAPRECORD}$$

Figure 75: Record Wrapper

With this rule, the following program has a type error when wrapping.

```
let r : { x : int } = { x = 0 ; y = 1 }
let _ = r.y
```

Alternatively, here is what wrapping might look like if we kept the undeclared labels:

$$\frac{v = \{\ell_1 = v_1; \dots; \ell_n = v_n; \ell'_1 = v'_1; \dots; \ell'_m = v'_m\} \quad \forall i. v_i : t_i \hookrightarrow v_i^w}{v : \{\ell_1 : t_1; \dots; \ell_n : t_n\} \hookrightarrow \{\ell_1 = v_1^w; \dots; \ell_n = v_n^w; \ell'_1 = v'_1; \dots; \ell'_m = v'_m\}}$$

5.3. Module wrapping

Like records, we wrap modules by cutting off extra labels. Notice that the module type has to be evaluated while the candidate member is wrapped. This is the first type extension for which wrappers may not evaluate. For example, the wrappers can fail (when the module type is ill formed) or vanish (when the module type itself has a type check inside it), so we get back a result r from the wrapper. We'll say that the non-values implicitly propagate, without writing the rules. The result r is not yet shown in the rule form.

$v : t \hookrightarrow v$ (module extensions)

$$\frac{\begin{array}{l} v = \text{struct let } \ell_1 = v_1 \dots \text{let } \ell_n = v_n \dots \text{end} \\ \forall i \in \{2, \dots, n\}. E\{\ell_1 \mapsto v_1^w\} \dots \{\ell_{i-1} \mapsto v_{i-1}^w\} \models \tau_i \Rightarrow t_i \\ \forall i \in \{1, \dots, n\}. v_i : t_i \hookrightarrow v_i^w \quad v' = \text{struct let } \ell_1 = v_1^w \dots \text{let } \ell_n = v_n^w \text{end} \end{array}}{v : \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{val } \ell_n : \tau_n \text{ end}, E \rangle \hookrightarrow v'} \text{WRAPMODULE}$$

Figure 76: Module Wrapper

We wrap the values when evaluating the module type itself. This ensures that if there is any misuse within the module type evaluation, then there is a reachable error during the process of wrapping. For example, the following program has an error *while wrapping*, but not while checking.

```
let M : sig
  val x : int
  val f : bool -> bool
  val y : if f x then int else bool
end = struct
  let x = 0
  let f = fun _ -> true
  let y = 1
end
```

5.4. Dependent function wrapping

Dependent function types can also wrap values. Here, we update the grammar for dependent function wrappers and follow it with the semantics.

We first extend the grammar to include values wrapped with dependent function types.

$$\begin{array}{l} w ::= \dots \\ \quad | \quad w(v, (x : t) \rightarrow \langle \tau, E \rangle) \text{ Dependent wrapped function} \end{array}$$

Figure 77: Dependent Functions Wrap Grammar Updates

The old WRAPFUN and REWRAPFUN rules are then overwritten to include dependent function types, too.

$v : t \hookrightarrow v$

$$\frac{v \text{ is not of the form } w \quad t \in \{t_1 \rightarrow t_2, (x : t_1) \rightarrow \langle \tau, E \rangle\}}{v : t \hookrightarrow W(v, t)} \text{WRAPFUN}$$

$$\frac{t \in \{t_1 \rightarrow t_2, (x : t_1) \rightarrow \langle \tau, E \rangle\}}{W(v, t') : t \hookrightarrow W(v, t)} \text{REWRAPFUN}$$

Figure 78: Function Wrap Updates

These rules simply ensure that there is never a doubly wrapped value and are otherwise agnostic to the value v . They are also agnostic to t (as long as it is a function), so they handle both the regular *and* dependent function type cases.

We must update the APPLWRAPCHECK rule to include dependent function types because the domain checking of a dependent function type is identical to that of a regular function type, so it can be combined with the old rule instead of being a new one.

$$\boxed{E \models e \Rightarrow r}$$

$$\frac{E \models e_1 \Rightarrow W(v, t) \quad v \in \{t_1 \rightarrow t_2, (x : t_1) \rightarrow \langle \tau, E' \rangle\} \quad E \models e_2 \Rightarrow v' \quad v' : t_1 \Downarrow \odot}{E \models e_1 \ e_2 \Rightarrow \odot} \text{APPLWRAPCHECK}$$

Figure 79: Check Wrapped Function Application

The other updates are for wrapped checkers and propagation of wrappers. To save space, we do not write the wrapped checkers, and we provide only one example rule of propagation.

It is easy to see in the following rule that it mirrors application with regular wrapping functions, but the codomain must be evaluated once the argument value is known.

$$\boxed{E \models e \Rightarrow r}$$

$$\frac{E \models e_1 \Rightarrow W(\langle \lambda x. e, E' \rangle, (x' : t_1) \rightarrow \langle \tau, E'' \rangle) \quad E \models e_2 \Rightarrow v \quad E'' \{x' \mapsto v\} \models \tau \Rightarrow t_2 \quad E' \{x \mapsto v\} \models e \Rightarrow v' \quad v' : t_2 \hookrightarrow v'_w}{E \models e_1 \ e_2 \Rightarrow v'_w} \text{APPLWRAPDEPFUN}$$

Figure 80: Apply Wrapped Dependent Functions

5.5. Other wrappers

The remaining wrappers are comparatively trivial, so we lump them under one section with little commentary.

$$\boxed{v : t \hookrightarrow v} \text{ (variant extensions)}$$

$$\frac{\ell = \ell_i \quad v : t_i \hookrightarrow v_w}{\ell v : \ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \hookrightarrow \ell v_w} \text{WRAPVARIANT}$$

Figure 81: Variant Wrapper

$$\boxed{v : t \hookrightarrow v} \text{ (polymorphism extensions)}$$

$$\frac{}{t : \text{type} \hookrightarrow t} \text{WRAPTYPE} \quad \frac{}{G(\alpha_n) : \alpha_n \hookrightarrow G(\alpha_n)} \text{WRAPPOLY}$$

Figure 82: Polymorphic Wrappers

$$\boxed{v : t \hookrightarrow v} \text{ (recursive types extensions)}$$

$$\frac{E \{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow t \quad v : t \hookrightarrow v_w}{v : \langle \mu x. \tau, E \rangle \hookrightarrow v_w} \text{WRAPMU}$$

$$\frac{E \{x \mapsto \langle \mu x. \tau, E \rangle\} \models \tau \Rightarrow \odot}{v : \langle \mu x. \tau, E \rangle \hookrightarrow \odot} \text{WRAPMUNONVALUE}$$

Figure 83: Wrap Recursive Types

Note that for refinements, because the value v is already known to be in the wrapping type, it passes the refinement and therefore already contains in itself all the information the refinement predicate might provide. Thus, the predicate can be tossed away.

$\boxed{v : t \hookrightarrow v}$ (refinement extensions)

$$\frac{v : t \hookrightarrow v_w}{v : \{x : t \mid \langle e, E \rangle\} \hookrightarrow v_w} \text{WRAPREFINEMENT}$$

Figure 84: Refinement Type Wrapper

$\boxed{v : t \hookrightarrow v}$ (unit extensions)

$$\frac{}{() : \text{unit} \hookrightarrow ()} \text{WRAPUNIT}$$

Figure 85: Wrap Unit

We cannot wrap a type with bottom. To explicitly demonstrate this, we say that wrapping errors, no matter the value.

$\boxed{v : t \hookrightarrow v}$ (bottom extensions)

$$\frac{}{v : \perp \hookrightarrow \mathbf{X}} \text{WRAPBOTTOM}$$

Figure 86: Wrap Bottom

6. Concolic

6.1. Concolic Steps

The above system is enough to refute typings, but it is not a systematic approach to *find* a refutation. We will augment it to be a concolic evaluator, which will be a set of rules that systematically finds a refutation if one exists. The symbolic component helps us build every possible proof tree, and the concrete component organizes that process in a manner that allows efficient refutations. A pure symbolic evaluator, on the other hand, may spend too much time searching shallow trees; a concolic evaluator builds full trees, one at a time.

We slowly augment the above base system to be a full concolic evaluator. The steps we take are as follows:

1. Add a symbolic component to concrete values.
2. Identify symbolic variables with timestamps.
3. Use an input environment to provide concrete values for some specific symbols.
4. Allow the input environment to decide which rules to take when several could apply.
5. Log the execution path.
6. Create a concolic loop that will run every execution path.

We will not show all the rules for each step. Instead, it is enough to provide a simple grammar change or extension and then show a few rules to demonstrate the most significant difference. Further, we only extend the most simple version of the system, at first. After we extend the full base system, we present the concolic version for several type extensions.

6.1.1. Concolic values

We update the grammar to include symbolic formulas, and the values are then concolic: some values have both a concrete and symbolic component.

symbols	$\varsigma ::=$	$(Identifiers)$
constants	$c ::=$	n Integer constants
		$ b$ Boolean constants
formulas	$s ::=$	c Constant formula
		$ \varsigma$ Symbol
		$ s \otimes s$ Symbolic operation
		$ \text{not } s$ Negated formula
values	$v ::=$	$\langle c, s \rangle$ Concolic value
		$ \langle \lambda x.e, E \rangle$ Function closure
		$ t$ Type value
		$ g$ Generated value

Figure 87: Simple Concolic Grammar Extensions

Everything else from the core grammar is unchanged; expressions and type values are all the same. It is only the integer and boolean values that have a symbolic component.

Then, operations on integers and booleans are changed to apply the meaning of the operation to the concrete components and to build a new formula with the symbolic components. Generated concolic values get a fresh symbol (which will be determinized later in this document). Anything that does not directly touch an integer or boolean is unaffected by this change. These updates are demonstrated in the following figures.

$E \models e \Rightarrow r$

 (partial)
$$\begin{array}{c}
\frac{E \models e_1 \Rightarrow \langle c_1, s_1 \rangle \quad E \models e_2 \Rightarrow \langle c_2, s_2 \rangle}{E \models e_1 \otimes e_2 \Rightarrow \langle c_1 \llbracket \otimes \rrbracket c_2, s_1 \otimes s_2 \rangle} \text{BINOP} \qquad \frac{E \models e_1 \Rightarrow \langle \text{true}, s \rangle \quad E \models e_2 \Rightarrow r}{E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFTRUE} \\
\\
\frac{E \models e_1 \Rightarrow \langle \text{false}, s \rangle \quad E \models e_3 \Rightarrow r}{E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFFALSE}
\end{array}$$

Figure 88: Evaluate simple concolic untyped expressions

$t \uparrow v$

 (partial)
$$\begin{array}{c}
\frac{b \in \mathbb{B} \quad \varsigma \text{ fresh}}{\text{bool} \uparrow \langle b, \varsigma \rangle} \text{GENBOOL} \qquad \frac{n \in \mathbb{Z} \quad \varsigma \text{ fresh}}{\text{int} \uparrow \langle n, \varsigma \rangle} \text{GENINT}
\end{array}$$

Figure 89: Simple concolic generators

$v : t \Downarrow \odot$

 (partial)
$$\begin{array}{c}
\frac{v \text{ is not } \langle c, s \rangle \text{ for } c \in \mathbb{Z}}{v : \text{int} \Downarrow \times} \text{REFUTEINT} \qquad \frac{v \text{ is not } \langle c, s \rangle \text{ for } c \in \mathbb{B}}{v : \text{bool} \Downarrow \times} \text{REFUTEBOOL} \\
\\
\frac{c \in \mathbb{Z}}{\langle c, s \rangle : \text{int} \Downarrow \checkmark} \text{CONFIRMIT} \qquad \frac{c \in \mathbb{B}}{\langle c, s \rangle : \text{bool} \Downarrow \checkmark} \text{CONFIRMBOL}
\end{array}$$

Figure 90: Simple concolic checkers

6.1.2. Timestamps

Just a “fresh” symbol is not very helpful because it is not predictable and therefore cannot be used. We will update the symbols to be identified by the current “time”. The time ticks as slow as is necessary; it only ticks when a symbolic value needs to be identified by the current time. Right now, it can be helpful to think of the time as “number of symbols made so far”.

The grammar is updated so that we have timestamps and symbols both identified by a simple integer. Then, when a value is the result of a computation, the time is also returned. When the result is a non-value (e.g. a type refutation), then the time is not needed because nothing else can happen: the non-value result propagates, and the time could not possibly be used for anything else. We thus set up the grammar to only return a time with value results.

$$\begin{array}{ll} \text{times } \pi ::= n & \text{Integers identify times} \\ \text{symbols } \varsigma ::= \pi & \text{Times identify symbols} \\ \text{evaluation results } r ::= v, \pi & \text{Value with time} \\ & | \odot \quad \text{Non-value result} \end{array}$$

Figure 91: Timestamped Concolic Grammar Extensions

We define $\pi^\triangleright = \pi + 1$ to be the next timestamp. $\pi_{\text{init}} = 0$ is the initial time.

We demonstrate how time is stateful with a few rules that do not immediately use the time. We then show how fresh symbols are made by referring to the current time.

$$\boxed{\pi, E \models e \Rightarrow r} \quad (\text{partial})$$

$$\begin{array}{c} \frac{\pi_0, E \models e_1 \Rightarrow \langle c_1, s_1 \rangle, \pi_1 \quad \pi_1, E \models e_2 \Rightarrow \langle c_2, s_2 \rangle, \pi_2}{\pi_0, E \models e_1 \otimes e_2 \Rightarrow \langle c_1 \llbracket \otimes \rrbracket c_2, s_1 \otimes s_2 \rangle, \pi_2} \text{BINOP} \\ \\ \frac{\pi_0, E \models e_1 \Rightarrow \langle \text{true}, s \rangle, \pi_1 \quad \pi_1, E \models e_2 \Rightarrow r}{\pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFTRUE} \\ \\ \frac{\pi_0, E \models e_1 \Rightarrow \langle \text{false}, s \rangle, \pi_1 \quad \pi_1, E \models e_3 \Rightarrow r}{\pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{IFFALSE} \end{array}$$

Figure 92: Evaluate timestamped concolic untyped expressions

$$\boxed{\pi \models t \uparrow v, \pi}$$

$$\begin{array}{c} \frac{b \in \mathbb{B} \quad \varsigma = \pi}{\pi \models \text{bool} \uparrow \langle b, \varsigma \rangle, \pi^\triangleright} \text{GENBOOL} \\ \\ \frac{n \in \mathbb{Z} \quad \varsigma = \pi}{\pi \models \text{int} \uparrow \langle n, \varsigma \rangle, \pi^\triangleright} \text{GENINT} \quad \frac{}{\pi \models t_1 \rightarrow t_2 \uparrow G(t_1 \rightarrow t_2), \pi} \text{GENFUN} \end{array}$$

Figure 93: Timestamped concolic generators

$$\boxed{\pi \models v : t \downarrow \odot}$$

$$\frac{v \text{ is not } \langle c, s \rangle \text{ for } c \in \mathbb{Z}}{\pi \models v : \text{int} \downarrow \mathbf{X}} \text{REFUTEINT} \quad \frac{c \in \mathbb{Z}}{\pi \models \langle c, s \rangle : \text{int} \downarrow \checkmark} \text{CONFIRMIT}$$

Figure 94: Timestamped concolic int checkers

While checkers don't produce an output time, they still need an input time, as demonstrated by a function checker, in case the generator or evaluation needs to know the time to make a symbol:

$$\boxed{\pi \models v : t \rightarrow t \Downarrow \odot} \text{ (partial)}$$

$$\frac{\pi_0 \models t_1 \Uparrow v, \pi_1 \quad \pi_1, E\{x \mapsto v\} \models e \Rightarrow v', \pi_2 \quad \pi_2 \models v' : t_2 \Downarrow \odot}{\pi_0 \models \langle \lambda x. e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKFUN}$$

Figure 95: Timestamped concolic $t \rightarrow t$ checker

6.1.3. Input environments

We now show how the timestamped symbols are helpful to determinize the generated integers and booleans. Above, we showed that we will identify fresh symbolic values by their times, and now we show how an input environment can use that time to decide the concrete value that is generated.

We simply extend the grammar with inputs and input environments:

$$\begin{array}{ll} \text{inputs } i & ::= c & \text{Concrete input} \\ \text{input environments } I & ::= \{\varsigma \mapsto i, \dots, \varsigma \mapsto i\} \end{array}$$

Figure 96: Input Environment Grammar Extensions

The input environment is read-only and is well-formed only if it is not a multimap. The evaluation relation is now $I, \pi, E \models e \Rightarrow r$, where we can read from input environment I . In this setup, it only affects the integer and boolean generation. We might write integer generation with a pair of rules, as follows:

$$\frac{n \in \mathbb{Z} \quad \forall n'. (\varsigma \mapsto n') \notin I \quad \varsigma = \pi}{I, \pi \models \text{int} \Uparrow \langle n, \varsigma \rangle, \pi^\triangleright} \text{GENINT'}$$

$$\frac{(\varsigma \mapsto n) \in I \quad \varsigma = \pi}{I, \pi \models \text{int} \Uparrow \langle n, \varsigma \rangle, \pi^\triangleright} \text{READINT'}$$

The first says that there is no mapping in the input environment for the current time, so we may generate an unconstrained integer. The second says that there *is* an entry in the input environment, so we read from it.

However, this can be shortened into one relation if we define $I(\varsigma) = \{i \mid (\varsigma \mapsto i) \in I\}$. So if $n \in \mathbb{Z}$ is an arbitrary integer and $\forall n'. (\varsigma \mapsto n') \notin I$, then $I(\varsigma) = \emptyset$, so $I(\varsigma) \subseteq \{n\}$. i.e. the premises of the first rule (GENINT') imply $I(\varsigma) \subseteq \{n\}$. Similarly, if $(\varsigma \mapsto n) \in I$, then $I(\varsigma) \subseteq \{n\}$ (supposing we have the restriction that I is not a multimap). Then the premises of the second rule (READINT') also imply $I(\varsigma) \subseteq \{n\}$.

Thus we can just write

$$\frac{n \in \mathbb{Z} \quad I(\varsigma) \subseteq \{n\} \quad \varsigma = \pi}{I, \pi \models \text{int} \Uparrow \langle n, \varsigma \rangle, \pi^\triangleright}$$

in place of the original two rules because it derives both of them. The new generator rules are in the following figure.

$$\boxed{I, \pi \models t \uparrow v, \pi}$$

$$\frac{b \in \mathbb{B} \quad I(\varsigma) \subseteq \{b\} \quad \varsigma = \pi}{I, \pi \models \text{bool} \uparrow \langle b, \varsigma \rangle, \pi^{\triangleright}} \text{GENBOOL}$$

$$\frac{n \in \mathbb{Z} \quad I(\varsigma) \subseteq \{n\} \quad \varsigma = \pi}{I, \pi \models \text{int} \uparrow \langle n, \varsigma \rangle, \pi^{\triangleright}} \text{GENINT} \quad \frac{}{I, \pi \models t_1 \rightarrow t_2 \uparrow G(t_1 \rightarrow t_2), \pi} \text{GENFUN}$$

Figure 97: Timestamped generators with input environment

6.1.4. Labels in input environments

It is not enough to just decide which inputs to give for symbolic integers and booleans. The system still has nondeterministic choice of rules. For example, we have the choice to type check a typed `let`-expression, or we can evaluate its body. The input environment should decide choice this, too.

We update the inputs to include “labels”, which will decide rule applications.

$$\begin{array}{ll} \text{labels } \ell ::= & (\text{Identifiers}) \\ \text{inputs } i ::= & \dots \\ & | \ell \quad \text{Rule label} \end{array}$$

Figure 98: Input Labels Grammar Extensions

The goal of these input labels is to restrict which rule can be used at a certain time. Much like how we read from the input environment when we generate concrete values, we read from the input environment to determinize the choice.

$$\boxed{I, \pi, E \models e \Rightarrow r} \quad (\text{partial})$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1 \quad I, \pi_1, E \models \tau \Rightarrow t, \pi_2 \quad I(\pi_2) \subseteq \{\text{EVAL}\} \quad I, \pi_2^{\triangleright}, E\{x \mapsto v\} \models e_2 \Rightarrow r}{I, \pi_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow r} \text{LETYPED}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1 \quad I, \pi_1, E \models \tau \Rightarrow t, \pi_2 \quad I(\pi_2) \subseteq \{\text{CHECK}\} \quad I, \pi_2^{\triangleright} \models v : t \Downarrow \odot}{I, \pi_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \odot} \text{LETCHECK}$$

Figure 99: Evaluate typed timestamped expressions

Above, the input environment restricts whether we type check or evaluate the body once the `let`-expression’s definition is evaluated. Similarly, it restricts how the checkers themselves work:

$$\boxed{I, \pi \models v : t \Downarrow \odot} \quad (\text{partial})$$

$$\frac{I(\pi_0) \subseteq \{\text{LEFT}\} \quad I, \pi_0^{\triangleright} \models t_1 \uparrow v, \pi_1 \quad I, \pi_1 \models v : t'_1 \Downarrow \odot}{I, \pi_0 \models G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKGENFUNLEFT}$$

$$\frac{I(\pi_0) \subseteq \{\text{RIGHT}\} \quad I, \pi_0^{\triangleright} \models t'_2 \uparrow v, \pi'_1 \quad I, \pi_1 \models v : t_2 \Downarrow \odot}{I, \pi_0 \models G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot} \text{CHECKGENFUNRIGHT}$$

Figure 100: Timestamped checkers

6.1.5. Path logging

Concolic evaluation is a process that runs all execution paths (and builds all symbolically equivalent trees), but in practice we do not want to actually construct a tree because it would be an unnecessarily large data structure. Instead, we can record a “path” that concisely represents what happened during evaluation. Paths are logged during executions, and they are the record of which branches were taken, as well as which labels were used at which time. The logged path can then be used to construct an input environment that directs evaluation down a *different* path the next time.

The path just needs to have enough information in it to allow us to construct this next input environment that derives a different path. The grammar is extended as follows.

path items	$u ::= s$	Singleton formula
	$ (\pi \mapsto \ell) (\ell, \dots, \ell)$	Timed label with alternatives
paths	$P ::= \varepsilon$	Empty path
	$ u; \dots; u$	Path item sequence

Figure 101: Paths Grammar Extensions

Paths are sequences of path items, which are one of two things:

- A true formula that is the symbolic component of a branch condition during the execution. The product of all the formulas in a logged path is always true under the assignment in the associated input environment.
- A label that was used, along with the other labels that *could* have been used, but were not. For example, $(\pi \mapsto \text{EVAL}) (\text{CHECK})$ means the execution chose to evaluate at time π , but another path could have checked, instead.

We make a path append operator, written with fat semicolon “ $;$ ”, which is associative and sequences the underlying path items. The empty path is the identity element under the sequencing operation, so $P = (\varepsilon; P) = (P; \varepsilon)$.

For example, when the condition evaluates to **false**, we have

$$\frac{I, \pi_0, E \models e_1 \Rightarrow \langle \text{false}, s \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_3 \Rightarrow r, P_3}{I, \pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r, P_1; \text{not } s; P_3}$$

In this case, we know that $s = \text{false}$ because s is the symbolic value for the concrete value **false**, and thus $(\text{not } s)$ is a **true** formula. The execution path of the entire expression is the execution path of the condition followed by the branch condition’s formula, and then the execution path of the body. Since this is the last extension to the rules, we leave the details of path logging to the full system, now that we have given this intuition.

Observe that there is a path with any result r , even if that result is a check result (⊙). This is because when the result is a confirmation (✓), we have not found a refutation (which is the goal), and we still need to search more paths; in order to do so, we need the path that led to the confirmation.

6.2. Base concolic system

6.2.1. Grammar

variables	x	$::=$	$(Identifiers)$	
labels	ℓ	$::=$	$(Identifiers)$	
integers	n	$::=$	$0 \mid 1 \mid 2 \mid \dots$	
booleans	b	$::=$	$\text{true} \mid \text{false}$	
times	π	$::=$	n	Integers identify times
symbols	ς	$::=$	π	Times identify symbols
constants	c	$::=$	n $\mid b$	
formulas	s	$::=$	c $\mid \varsigma$ $\mid s \otimes s$ $\mid \text{not } s$	Constant formula Symbol Symbolic operation Negated formula
values	v	$::=$	$\langle c, s \rangle$ $\mid \langle \lambda x.e, E \rangle$ $\mid t$ $\mid g$	Concolic value Function closure Type value Generated value
type values	t	$::=$	int $\mid \text{bool}$ $\mid t \rightarrow t$	Function type value
generated values	g	$::=$	$G(t \rightarrow t)$	Generated function
expressions	e	$::=$	τ $\mid x$ $\mid n$ $\mid b$ $\mid \lambda x.e$ $\mid e e$ $\mid e \otimes e$ $\mid \text{if } e \text{ then } e \text{ else } e$ $\mid \text{let } x = e \text{ in } e$ $\mid \text{let } x : \tau = e \text{ in } e$	Type as expression Arbitrary binary operation Untyped let-expression Typed let-expression
type expressions	τ	$::=$	e $\mid \text{int}$ $\mid \text{bool}$ $\mid \tau \rightarrow \tau$	Expression as type Function type expression
binary operations	$\otimes$	$::=$	$+$ $ $ $-$ $ $ $*$ $ $ $/$ $ $ $==$	
type check result	$\odot$	$::=$	$\mathbf{X}$ $\mid \checkmark$	Refutation Confirmation
environments	E	$::=$	$\{x \mapsto v, \dots, x \mapsto v\}$	
inputs	i	$::=$	c $\mid \ell$	Concrete input Rule label
input environments	I	$::=$	$\{\varsigma \mapsto i, \dots, \varsigma \mapsto i\}$	
path items	u	$::=$	s $\mid (\pi \mapsto \ell) (\ell, \dots, \ell)$	Singleton formula Timed label with alternatives
paths	P, Q	$::=$	ε $\mid u; \dots; u$	Empty path Path item sequence
evaluation result	r	$::=$	v, π $\mid \odot$	Value with time Non-value

Figure 102: Concolic Grammar

As before, we define $\pi^\triangleright = \pi + 1$ to be the next timestamp. Timestamps are used to identify symbolic inputs; by construction, each input corresponds to exactly one timestamp. $\pi_{\text{init}} = 0$ is the initial time.

Input environments are well formed only if they are not multimaps.

The path append operator “;” is the same as above: it is associative, and ε is the identity element.

6.2.2. Rules

Commentary in this section mainly pertains to paths and natural language reading of the relations. Otherwise the system should be clear and familiar by now.

$I, \pi, E \models e \Rightarrow r, P$ reads “with input environment I , beginning at time π in the environment E , the expression e concolically evaluates to the result r along path P ”.

$I, \pi, E \models e \Rightarrow r, P$

 (1 of 5)
$$\begin{array}{c}
\frac{(x \mapsto v) \in E}{I, \pi, E \models x \Rightarrow v, \pi, \varepsilon} \text{VARIABLE} \qquad \frac{}{I, \pi, E \models \lambda x. e \Rightarrow \langle \lambda x. e, E \rangle, \pi, \varepsilon} \text{CLOSURE} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow \langle c_1, s_1 \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow \langle c_2, s_2 \rangle, \pi_2, P_2}{I, \pi_0, E \models e_1 \otimes e_2 \Rightarrow \langle c_1 \llbracket \otimes \rrbracket c_2, s_1 \otimes s_2 \rangle, \pi_2, P_1 ; P_2} \text{BINOP} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow \langle \lambda x. e, E' \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow v, \pi_2, P_2 \quad I, \pi_2, E' \{x \mapsto v\} \models e \Rightarrow r, P_3}{I, \pi_0, E \models e_1 e_2 \Rightarrow r, P_1 ; P_2 ; P_3} \text{APPL} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow \langle \text{true}, s \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow r, P_2}{I, \pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r, P_1 ; s ; P_2} \text{IFTRUE} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow \langle \text{false}, s \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_3 \Rightarrow r, P_3}{I, \pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r, P_1 ; \text{not } s ; P_3} \text{IFFALSE} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P_1 \quad I, \pi_1, E \{x \mapsto v\} \models e_2 \Rightarrow r, P_2}{I, \pi_0, E \models \text{let } x = e_1 \text{ in } e_2 \Rightarrow r, P_1 ; P_2} \text{LET}
\end{array}$$

Figure 103: Evaluate untyped expressions

$I, \pi, E \models e \Rightarrow r, P$

 (2 of 5)
$$\begin{array}{c}
\frac{}{I, \pi, E \models \text{int} \Rightarrow \text{int}, \pi, \varepsilon} \text{INT} \qquad \frac{}{I, \pi, E \models \text{bool} \Rightarrow \text{bool}, \pi, \varepsilon} \text{BOOL} \\
\\
\frac{I, \pi_0, E \models \tau_1 \Rightarrow t_1, \pi_1, P_1 \quad I, \pi_1, E \models \tau_2 \Rightarrow t_2, \pi_2, P_2}{I, \pi_0, E \models \tau_1 \rightarrow \tau_2 \Rightarrow t_1 \rightarrow t_2, \pi_2, P_1 ; P_2} \text{FUNTYPE}
\end{array}$$

Figure 104: Evaluate types

Generators currently never produce a non-empty path, nor do they ever fail to evaluate, but with some extensions, they will, so we set up the infrastructure for it to do so.

$I, \pi \models t \uparrow v, \pi', P$ reads “with input environment I , beginning at time π , the type value t generates the concolic value v with final time π' along the path P ”.

$$\boxed{I, \pi \models t \uparrow r, P}$$

$$\frac{b \in \mathbb{B} \quad I(\varsigma) \subseteq \{b\} \quad \varsigma = \pi}{I, \pi \models \text{bool} \uparrow \langle b, \varsigma \rangle, \pi^\triangleright, \varepsilon} \text{GENBOOL}$$

$$\frac{n \in \mathbb{Z} \quad I(\varsigma) \subseteq \{n\} \quad \varsigma = \pi}{I, \pi \models \text{int} \uparrow \langle n, \varsigma \rangle, \pi^\triangleright, \varepsilon} \text{GENINT} \quad \frac{}{I, \pi \models t_1 \rightarrow t_2 \uparrow G(t_1 \rightarrow t_2), \pi, \varepsilon} \text{GENFUN}$$

Figure 105: Generators

$I, \pi \models v : t \Downarrow \odot, P$ reads “with input environment I , beginning at time π , the result of checking that value v has type t is $\odot$ along path P ”

$$\boxed{I, \pi \models v : t \Downarrow \odot, P} \quad (1 \text{ of } 2)$$

$$\frac{v \text{ is not } \langle c, s \rangle \text{ for } c \in \mathbb{Z}}{I, \pi \models v : \text{int} \Downarrow \mathbf{X}, \varepsilon} \text{REFUTEINT} \quad \frac{v \text{ is not } \langle c, s \rangle \text{ for } c \in \mathbb{B}}{I, \pi \models v : \text{bool} \Downarrow \mathbf{X}, \varepsilon} \text{REFUTEBOOL}$$

$$\frac{c \in \mathbb{Z}}{I, \pi \models \langle c, s \rangle : \text{int} \Downarrow \checkmark, \varepsilon} \text{CONFIRMIT} \quad \frac{c \in \mathbb{B}}{I, \pi \models \langle c, s \rangle : \text{bool} \Downarrow \checkmark, \varepsilon} \text{CONFIRMBOL}$$

$$\frac{I, \pi_0 \models t_1 \uparrow v, \pi_1, P_1 \quad I, \pi_1, E\{x \mapsto v\} \models e \Rightarrow v', \pi_2, P_2 \quad I, \pi_2 \models v' : t_2 \Downarrow \odot, P_3}{I, \pi_0 \models \langle \lambda x. e, E \rangle : t_1 \rightarrow t_2 \Downarrow \odot, P_1 ; P_2 ; P_3} \text{CHECKFUN}$$

$$\frac{I(\pi_0) \subseteq \{\text{LEFT}\} \quad I, \pi_0^\triangleright \models t_1 \uparrow v, \pi_1, P_1 \quad I, \pi_1 \models v : t'_1 \Downarrow \odot, P_2}{I, \pi_0 \models G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot, (\pi_0 \mapsto \text{LEFT}) (\text{RIGHT}) ; P_1 ; P_2} \text{CHECKGENFUNLEFT}$$

$$\frac{I(\pi_0) \subseteq \{\text{RIGHT}\} \quad I, \pi_0^\triangleright \models t'_2 \uparrow v, \pi_1, P_1 \quad I, \pi_1 \models v : t_2 \Downarrow \odot, P_2}{I, \pi_0 \models G(t'_1 \rightarrow t'_2) : t_1 \rightarrow t_2 \Downarrow \odot, (\pi_0 \mapsto \text{RIGHT}) (\text{LEFT}) ; P_1 ; P_2} \text{CHECKGENFUNRIGHT}$$

Figure 106: Checkers

$$\boxed{I, \pi, E \models e \Rightarrow r, P} \quad (3 \text{ of } 5)$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P_1 \quad I, \pi_1, E \models \tau \Rightarrow t, \pi_2, P_2 \quad I(\pi_2) \subseteq \{\text{EVAL}\} \quad I, \pi_2^\triangleright, E\{x \mapsto v\} \models e_2 \Rightarrow r, P_3}{I, \pi_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow r, P_1 ; P_2 ; (\pi_2 \mapsto \text{EVAL}) (\text{CHECK}) ; P_3} \text{LET TYPED}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P_1 \quad I, \pi_1, E \models \tau \Rightarrow t, \pi_2, P_2 \quad I(\pi_2) \subseteq \{\text{CHECK}\} \quad I, \pi_2^\triangleright \models v : t \Downarrow \odot, P_3}{I, \pi_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \odot, P_1 ; P_2 ; (\pi_2 \mapsto \text{CHECK}) (\text{EVAL}) ; P_3} \text{LET CHECK}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P_1 \quad I, \pi_1, E \models \tau \Rightarrow v', \pi_2, P_2 \quad v' \text{ is not of the form } t}{I, \pi_0, E \models \text{let } x : \tau = e_1 \text{ in } e_2 \Rightarrow \mathbf{X}, P_1 ; P_2} \text{FAILTYPEEVAL}$$

Figure 107: Evaluate Typed Let-Expressions

In the above rules for **let**-expressions, the timed label is logged in the path as late as possible, after the type has been evaluated; if the type fails to evaluate, then there is no label to log, so it does not make much sense to go earlier.

$I, \pi, E \models e \Rightarrow r, P$ (4 of 5)

$$\frac{I, \pi_0, E \models e_1 \Rightarrow G(t_1 \rightarrow t_2), \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow v, \pi_2, P_2 \quad I(\pi_2) \subseteq \{\text{EVAL}\} \quad I, \pi_2^\triangleright \models t_2 \uparrow v', \pi_3, P_3}{I, \pi_0, E \models e_1 \ e_2 \Rightarrow v', \pi_3, P_1 ; P_2 ; (\pi_2 \mapsto \text{EVAL}) (\text{CHECK}) ; P_3} \text{APPLGENFUN}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow G(t_1 \rightarrow t_2), \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow v, \pi_2, P_2 \quad I(\pi_2) \subseteq \{\text{CHECK}\} \quad I, \pi_2^\triangleright \models v : t_1 \downarrow \circlearrowleft, P_3}{I, \pi_0, E \models e_1 \ e_2 \Rightarrow \circlearrowleft, P_1 ; P_2 ; (\pi_2 \mapsto \text{CHECK}) (\text{EVAL}) ; P_3} \text{APPLCHECK}$$

Figure 108: Apply to Generated Functions

$I, \pi, E \models e \Rightarrow r, P$ (5 of 5)

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v_1, \pi_1, P_1 \quad v_1 \text{ is not a valid operand with } \llbracket \otimes \rrbracket}{I, \pi_0, E \models e_1 \otimes e_2 \Rightarrow \mathbf{X}, P_1} \text{FAILBINOP1}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v_1, \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow v_2, \pi_2, P_2 \quad v_1 \text{ and } v_2 \text{ are not valid operands with } \llbracket \otimes \rrbracket}{I, \pi_0, E \models e_1 \otimes e_2 \Rightarrow \mathbf{X}, P_1 ; P_2} \text{FAILBINOP2}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P \quad v \text{ is of the form } t \text{ or } \langle c, s \rangle}{I, \pi_0, E \models e_1 \ e_2 \Rightarrow \mathbf{X}, P} \text{FAILAPPL}$$

$$\frac{I, \pi_0, E \models e_1 \Rightarrow v, \pi_1, P \quad v \text{ is not of the form } \langle b, s \rangle \text{ for } b \in \mathbb{B}}{I, \pi_0, E \models \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow \mathbf{X}, P} \text{FAILIF}$$

$$\frac{I, \pi_0, E \models \tau_1 \Rightarrow v, \pi_1, P \quad v \text{ is not of the form } t}{I, \pi_0, E \models \tau_1 \rightarrow \tau_2 \Rightarrow \mathbf{X}, P} \text{FAILFUNTYPE1}$$

$$\frac{I, \pi_0, E \models \tau_1 \Rightarrow t, \pi_1, P_1 \quad I, \pi_1, E \models \tau_2 \Rightarrow v, \pi_2, P_2 \quad v \text{ is not of the form } t}{I, \pi_0, E \models \tau_1 \rightarrow \tau_2 \Rightarrow \mathbf{X}, P_1 ; P_2} \text{FAILFUNTYPE2}$$

Figure 109: Type mismatches

$I, \pi \models v : t \downarrow \circlearrowleft, P$ (2 of 2)

$$\frac{I, \pi_0 \models t_1 \uparrow v, \pi_1, P_1 \quad I, \pi_1, E\{x \mapsto v\} \models e \Rightarrow \circlearrowleft, P_2}{I, \pi_0 \models \langle \lambda x. e, E \rangle : t_1 \rightarrow t_2 \downarrow \circlearrowleft, P_1 ; P_2} \text{CHECKFUNNONVALUE}$$

Figure 110: Checker via non-evaluation

As in the non-concolic system, check results propagate through the evaluation of expressions, but these rules are elided for brevity. Here are two examples to demonstrate what is meant here:

$$\begin{array}{c}
\frac{I, \pi_0, E \models e_1 \Rightarrow \langle c_1, s_1 \rangle, \pi_1, P_1 \quad I, \pi_1, E \models e_2 \Rightarrow \odot, P_2}{I, \pi_0, E \models e_1 \otimes e_2 \Rightarrow \odot, P_1 ; P_2} \text{PROPAGATEBINOP} \\
\\
\frac{I, \pi_0, E \models e_1 \Rightarrow \odot, P}{I, \pi_0, E \models e_1 \quad e_2 \Rightarrow \odot, P} \text{PROPAGATEAPPL}
\end{array}$$

6.2.3. Concolic loop

The concolic loop is the process through which the program is repeatedly evaluated in every possible way. It, too, is a set of inference rules, which produce a concolic “answer”.

The loop maintains a set of targets, which are derived from paths and are used to direct evaluation down a new path prefix. We extend the grammar to represent targets, sets of targets, and answers from the concolic loop.

target units	$\nu ::= s$	Formula
	$\mid \pi \mapsto \ell$	Timed label
targets	$T ::= \{\nu, \dots, \nu\}$	
target sets	$\mathbb{T} ::= \{T, \dots, T\}$	
concolic answers	$a ::= \text{ok}$	No error found
	$\mid \text{unknown}$	Inconclusive result
	$\mid \text{error}$	Error found

Figure 111: Concolic Loop Grammar Extensions

Targets are sets of formulas and timed labels. The formulas are some necessary conditions on the symbolic components of values during evaluation. If an input environment I is a “solution” to the formulas in target T , then an evaluation under I has an execution path with some prefix made of exactly the formulas and timed labels in T . For this reason, we say a target *describes* a path. Targets are unordered, but it may help with intuition to think of them as paths without alternative labels.

$\mathbb{T} \models e \Rightarrow a$ reads “with set of not-yet-run targets $\mathbb{T}$, running all of them through the expression e , and any more targets encountered along the way, will produce the concolic answer a ”. This is a bit hard to phrase in English, and hopefully the rules are more clear.

We kickstart the concolic loop with $\{\varepsilon\} \models e \Rightarrow a$ to get the concolic answer a for an expression e . i.e. the first target is the empty path, which is trivially satisfiable with the empty input environment: any input environment will run a path whose prefix is the empty path.

The following rules use functions $\text{TARGETS}(T, P)$ and $\text{SOLVE}(T)$, which are defined below the rules. Intuitively, $\text{TARGETS}(T, P)$ are the targets derived from an evaluation taking path P that was directed to hit T , and $\text{SOLVE}(T)$ is an input environment that directs evaluation to hit T .

$$\boxed{\mathbb{T} \models e \Rightarrow a}$$

$$\begin{array}{c}
\frac{}{\emptyset \models e \Rightarrow \text{ok}} \text{OK} \\
\\
\frac{I = \text{SOLVE}(T) \quad I, \pi_{\text{init}}, \emptyset \models e \Rightarrow v, \pi, P \quad \mathbb{T} \cup \text{TARGETS}(T, P) \models e \Rightarrow a}{\mathbb{T} \sqcup \{T\} \models e \Rightarrow a} \text{VALUE} \\
\\
\frac{I = \text{SOLVE}(T) \quad I, \pi_{\text{init}}, \emptyset \models e \Rightarrow \checkmark, P \quad \mathbb{T} \cup \text{TARGETS}(T, P) \models e \Rightarrow a}{\mathbb{T} \sqcup \{T\} \models e \Rightarrow a} \text{VANISH} \\
\\
\frac{I = \text{SOLVE}(T) \quad I, \pi_{\text{init}}, \emptyset \models e \Rightarrow \mathbf{X}, P}{\mathbb{T} \sqcup \{T\} \models e \Rightarrow \text{error}} \text{ERROR} \qquad \frac{\text{SOLVE}(T) = \text{unsat} \quad \mathbb{T} \models e \Rightarrow a}{\mathbb{T} \sqcup \{T\} \models e \Rightarrow a} \text{UNSAT} \\
\\
\frac{\text{SOLVE}(T) = \text{timeout} \quad \mathbb{T} \models e \Rightarrow a}{\mathbb{T} \sqcup \{T\} \models e \Rightarrow \min(\text{unknown}, a)} \text{UNKNOWN}
\end{array}$$

Figure 112: Concolic loop

The concolic answers are ordered from smallest to largest (where the ordering is used by $\min$) as follows: **error**, **unknown**, **ok**. This means any **unknown** trumps **ok** as the returned answer from the **UNKNOWN** rule, but **error** wins all.

Disjoint union ($\sqcup$) is used to say that $\mathbb{T}$ does not contain T , effectively removing the present target T from the running set of targets.

Definition 1 (ASSIGN): For set of formulas $S = \{s_1, \dots, s_n\}$, we say $\text{ASSIGN}(S)$ is a canonical satisfying assignment to the symbols in the satisfiable formulas $\{s_1, \dots, s_n\}$. It is of the form I and is “canonical” in the sense that it is one particular concrete assignment, even when there are many possible satisfying assignments.

Definition 2 (SOLVE): For target T , we say $\text{SOLVE}(T)$ is the input environment that directs an evaluation down the path described by T , if possible. Specifically, it is:

- $\text{SOLVE}(T) = \text{unsat}$ if $\{s \mid s \in T\}$ is unsatisfiable (no evaluation can take the path described by T).
- $\text{SOLVE}(T) = \text{ASSIGN}(\{s \mid s \in T\}) \cup \{\pi \mapsto \ell \mid \pi \mapsto \ell \in T\}$ otherwise.

In practice, we say $\text{SOLVE}(T) = \text{timeout}$ when an SMT solver takes too long to return either of the above answers, but it is not mathematically necessary.

Definition 3 (TARGETS): We define $\text{TARGETS}(T, P)$ to be the set of all target paths discovered during an evaluation. Intuitively, these represent branches not taken during evaluation and not seen in any prior evaluation—they describe the neighboring paths to P that come *after* the path prefix described by target T .

Formally, $\text{TARGETS}(T, P)$ is well-defined only when $|T| \leq |P|$, where $|P|$ is the number of path items u in P . Say $|T| = n$, and $P = P_T ; P'$ for $|P_T| = n$. Then

- $\text{TARGETS}(T, P_T ; \varepsilon) = \emptyset$.
- $\text{TARGETS}(T, P) = \text{TARGETS}(T \cup \{s\}, P) \cup \{T \cup \{\text{not } s\}\}$ when $P = P_T ; s ; P''$.
- $\text{TARGETS}(T, P) = \text{TARGETS}(T \cup \{\pi \mapsto \ell\}, P) \cup \{T \cup \{\pi \mapsto \ell_i\} \mid i \in \{1, \dots, n\}\}$ when $P = P_T ; (\pi \mapsto \ell) (\ell_1, \dots, \ell_n) ; P''$

6.2.4. Efficiency of concolic evaluation in practice

Here we comment on how to make the concolic process more efficient. There are several things to notice, and we address each of them, mostly informally.

- Solving efficiency with respect to labels.
- Solving efficiency with respect to formulas.
- Prioritization of targets.
- Continuation reuse.

We address these in order.

6.2.4.1. Solving labels

One can notice that it is always satisfiable to use an alternative label. That is, for all $T' \in \{T \cup \{\pi \mapsto \ell_i\} \mid i \in \{1, \dots, n\}\}$ in the definition of $\text{TARGETS}(T, P)$, we know T' is satisfiable if P came from an evaluation: its solution is simply the input environment which was used by the evaluation that produced the path P —recall that the product of formulas in a path is a **true** formula satisfied by the inputs used.

Hence, if the inputs are logged (recall that some are unplanned, and hence the input environment passed into the evaluation is not *all* inputs used), then the inputs may be extended by the label ℓ_i for each i , and we have a solution to the target.

6.2.4.2. Solving formulas

We noticed above that alternative labels can always be used because they are effectively disconnected from the formulas. In a similar vein, we can see that only the formulas that (transitively) share a symbol with the target formula need to be solved. The rest are disconnected, and they are trivially solved by the logged input environment, which can then be extended with the solution to the other, independent constraints.

Therefore, when we make a target with $\{T \cup \{\text{not } s\}\}$, we will compute the strongly connected component of s within the formulas of T , and only solve that connected component. The solution to the rest of the formulas is trivial.

This idea was first documented in the paper EXE: Automatically Generating Inputs of Death by Cadar et al.

6.2.4.3. Target prioritization

There has been lots of research into how to prioritize targets in the path search, with strategies including DFS, BFS, uniform random search, MCTS, coverage-guided search, fitness functions, and more. We can spend endless time optimizing target selection, or we can notice that for the purposes of type checking, errors often show up at base cases, and breadth-first-search is performant. It avoids the pitfalls of depth-first-search, and it is vastly simpler than the advanced alternatives.

We thus pop paths from the target set in order of the number of formulas they have; our search is breadth first with respect to the symbolic branching depth of paths.

6.2.4.4. Continuation reuse

The above concolic loop specifies that the program is rerun from the beginning with every new input environment. This does unnecessary work when the input environment only differs from a previous one by a single timed label. Instead, when the target is a timed label, a continuation can be saved alongside the target, and that continuation is called instead of running the program from the beginning. In fact, it seems to be performant to do this *during* the main evaluation

instead of creating a target at all. For example, when a typed `let`-expression is evaluated, we first run the checker, and then we evaluate the definition. The resulting paths are not sequenced because that would create unnecessary work. Instead, we take special care to maintain several distinct and independent logged paths and evaluation results.

This reduces the number of targets added to the target set and avoids rerunning the program from the start. It is future work to save all targets with a continuation, which would require knowing the continuation as of the smallest symbol in the target formula's connected component. This sounds expensive and possibly more work than it's worth.

6.3. Concolic Extensions

Here we provide a few concolic rules for the type extensions that have interesting concolic semantics.

6.3.1. Concolic Records

Timed labels are used to pick one rule from a choice of rules. In all cases so far, there has only been a pair of rules to choose from. With records, however, there are arbitrarily many labels to check, and hence arbitrarily many rules that can apply. The concolic record checking rules log all of the alternative labels to the path when a label is checked.

$$\boxed{I, \pi \models v : t \Downarrow \odot, P} \quad (\text{concolic records extensions})$$

$$\frac{v \text{ is not a record}}{I, \pi \models v : \{\ell_1 : t_1; \dots; \ell_n : t_n\} \Downarrow \mathbf{X}, \varepsilon} \text{REFUTENONRECORD}$$

$$\frac{\{\ell_1, \dots, \ell_n\} \not\subseteq \{\ell'_1, \dots, \ell'_m\}}{I, \pi \models \{\ell'_1 = v_1; \dots; \ell'_m = v_m\} : \{\ell_1 : t_1; \dots; \ell_n : t_n\} \Downarrow \mathbf{X}, \varepsilon} \text{REFUTEMISSINGLABEL}$$

$$\frac{
\begin{array}{c}
v = \{\ell'_1 = v_1; \dots; \ell'_m = v_m\} \\
\{\ell_1, \dots, \ell_n\} \subseteq \{\ell'_1, \dots, \ell'_m\} \quad I(\pi) \subseteq \{\ell_j\} \quad \ell'_i = \ell_j \quad I, \pi^\triangleright \models v_i : t_j \Downarrow \odot, P
\end{array}
}{I, \pi \models v : \{\ell_1 : t_1; \dots; \ell_n : t_n\} \Downarrow \odot, (\pi \mapsto \ell_j) (\ell_1, \dots, \ell_{j-1}, \ell_{j+1}, \dots, \ell_n); P} \text{CHECKRECORDLABEL}$$

Figure 113: Concolic Record Checkers

We say that the check took the path using label ℓ_j , and the alternatives are all the labels in the record type except ℓ_j . Considering the efficiency remarks in Section 6.2.4.4, these alternative record checks would all be done immediately, and they would not be added to the target queue.

6.3.2. Concolic Modules

Concolic modules are like concolic records but with extra paths from evaluating the type itself when checking, and the first label gets special consideration because it is already evaluated to a type value.

$I, \pi \models v : t \Downarrow \odot, P$ (concolic module extensions)

$$\begin{array}{c}
\frac{v \text{ is not a module}}{I, \pi \models v : \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \mathbf{X}, \varepsilon} \text{REFUTENONMODULE} \\
\\
\frac{v = \text{struct let } \ell'_1 = v_1 \dots \text{ let } \ell'_m = v_m \text{ end} \quad \{\ell_1, \dots, \ell_n\} \not\subseteq \{\ell'_1, \dots, \ell'_m\}}{I, \pi \models v : \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \Downarrow \mathbf{X}, \varepsilon} \text{REFUTEMISSINGMODULELABEL} \\
\\
\frac{v = \text{struct let } \ell_1 = v_1 \dots \text{ let } \ell_n = v_n \dots \text{ end} \quad t = \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \quad I(\pi) \subseteq \{\ell_1\} \quad I, \pi^\triangleright \models v_1 : t_1 \Downarrow \odot, P}{I, \pi \models v : t \Downarrow \odot, (\pi \mapsto \ell_1) (\ell_2, \dots, \ell_n) ; P} \text{CHECKFIRSTMODULELABEL} \\
\\
\frac{v = \text{struct let } \ell_1 = v_1 \dots \text{ let } \ell_n = v_n \dots \text{ end} \quad t = \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \quad i \in \{2, \dots, n\} \quad I(\pi) \subseteq \{\ell_i\} \quad I, \pi^\triangleright, E\{\ell_1 \mapsto v_1\} \dots \{\ell_{i-1} \mapsto v_{i-1}\} \models \tau_i \Rightarrow \pi', t_i, P \quad I, \pi' \models v_i : t_i \Downarrow \odot, P'}{I, \pi \models v : t \Downarrow \odot, (\pi \mapsto \ell_i) (\ell_1, \dots, \ell_{i-1}, \ell_{i+1}, \dots, \ell_n) ; P ; P'} \text{CHECKMODULELABEL}
\end{array}$$

Figure 114: Concolic Module Checkers

Also, since generation of a module can fail to produce a value, the $I, \pi \models t \Uparrow r, P$ relation actually can produce a result r instead of a value v and time π . For brevity, we do not write out the failure case, but we do show a successful concolic module generation.

$I, \pi \models t \Uparrow r, P$ (concolic module extensions)

$$\frac{t = \langle \text{sig val } \ell_1 : t_1 \text{ val } \ell_2 : \tau_2 \dots \text{ val } \ell_n : \tau_n \text{ end}, E \rangle \quad \forall i \in \{1, \dots, n\}. I, \pi_i \models t_i, v_i \Uparrow \pi'_i, P'_i \quad \forall i \in \{2, \dots, n\}. I, \pi'_{i-1}, E\{\ell_1 \mapsto v_1\} \dots \{\ell_{i-1} \mapsto v_{i-1}\} \models \tau_i \Rightarrow t_i, \pi_i, P_i}{I, \pi_1 \models t \Uparrow \text{struct let } \ell_1 = v_1 \dots \text{ let } \ell_n = v_n \text{ end}, \pi'_n, P'_1 ; P_1 ; \dots ; P'_n ; P_n} \text{GENMODULE}$$

Figure 115: Concolic Module Generator

6.3.3. Concolic Variants

As the dual to records, variant types have generators with a nondeterministic choice of rules. Thus, the concolic rule for the variant generator reads from the input environment.

$I, \pi \models t \Uparrow r, P$ (concolic variant extensions)

$$\frac{I(\pi) \subseteq \{\ell_i\} \quad I, \pi^\triangleright \models t_i \Uparrow v_i, \pi', P}{I, \pi \models \ell_1 \text{ of } t_1 \mid \dots \mid \ell_n \text{ of } t_n \Uparrow \ell_i v_i, \pi', (\pi \mapsto \ell_i) (\ell_1, \dots, \ell_{i-1}, \ell_{i+1}, \dots, \ell_n) ; P} \text{GENVARIANT}$$

Figure 116: Concolic Variant Generator

Contrary to record checking, we do *not* want to run the associated targets immediately (for each $\ell_j \neq \ell_i$) because generated values escape and are used in the rest of the arbitrarily large program. To try many generated values at once would be expensive, and we prefer the narrow

search of trying one at a time. We can solve these targets easily, but we do actually want to push them all to the target set.

6.3.4. Concolic Refinement Types

Concolic refinement types present some neat scenarios because of the symbolic component of the predicate, so we discuss them here.

We observe that there is no reason to want the predicate to evaluate to **false** when generating. In such a case, generation simply vanishes and is pointless; it is no help in finding a refutation. Similarly, we do not care if the predicate evaluates to **true** when checking. Therefore, when the predicate is **true** when generating and **false** when checking, we need to log the formula as a branch condition, but we do *not* want to make a target out of its negation. To do this, we extend the path item grammar with formulas that should not be flipped.

path items $u ::= \dots$
 $\quad \mid \underline{s} \text{ Nonflipping formula}$

Figure 117: Concolic Refinement Grammar Extensions

Then, the TARGETS definition is extended with the following case to *not* make a target from (not s) when $\underline{s}$ is in the path.

- $\text{TARGETS}(T, P) = \text{TARGETS}(T \cup \{s\}, P)$ when $P = P_T ; \underline{s} ; P''$.

Now, when the generated candidate passes the predicate, we log $\underline{s}$.

$I, \pi \models t \uparrow r, P$ (concolic refinement extensions)

$$\frac{I, \pi_0 \models t \uparrow v, \pi_1, P_1 \quad I, \pi_1, E\{x \mapsto v\} \models e \Rightarrow \langle \text{true}, s \rangle, \pi_2, P_2}{I, \pi_0 \models \{x : t \mid \langle e, E \rangle\} \uparrow v, \pi_2, P_1 ; P_2 ; \underline{s}} \text{GENREFINEMENT}$$

$$\frac{I, \pi_0 \models t \uparrow v, \pi_1, P_1 \quad I, \pi_1, E\{x \mapsto v\} \models e \Rightarrow \langle \text{false}, s \rangle, \pi_2, P_2}{I, \pi_0 \models \{x : t \mid \langle e, E \rangle\} \uparrow \checkmark, P_1 ; P_2 ; \text{not } s} \text{GENREFINEMENTVANISH}$$

$$\frac{I, \pi_0 \models t \uparrow v, \pi_1, P_1 \quad I, \pi_1, E\{x \mapsto v\} \models e \Rightarrow \odot, P_2}{I, \pi_0 \models \{x : t \mid \langle e, E \rangle\} \uparrow \odot, P_1 ; P_2} \text{GENREFINEMENTNONEVAL}$$

$$\frac{I, \pi_0 \models t \uparrow \odot, P_1}{I, \pi_0 \models \{x : t \mid \langle e, E \rangle\} \uparrow \odot, P_1} \text{GENREFINEMENTNONVALUE}$$

Figure 118: Concolic Refinement Type Generator

When generating, it might be tempting to not solve for all the ways for the predicate to come out **true**. That is, we might think about fixing everything in the path P_2 , saying that nothing in it should be negated. We already passed the refinement, so why bother finding out how else it could come out true? It's because the predicate evaluating to **true** may pin down some characteristics of the candidate v that we might like to be flexible. E.g.

```
let t = { x : int | let _ = if x > 0 then 0 else 1 in true }
```

All integers pass this predicate, but depending on the first random choice we make, fixing (but not allowing solves within) the path P_2 would either force all members to be positive or all members to be non-positive. We cannot remedy this by just not logging P_2 at all because we

need to have something that remembers the predicate evaluated to `true`, or else there may be usage errors later (in case the predicate actually does something, which the above does not).

When checking refinement types in the concolic system, we can use the `CHECK` and `EVAL` labels to choose whether we do the predicate or base type.

$$\boxed{v:t \Downarrow \odot} \text{ (refinement extensions)}$$

$$\frac{I(\pi) \subseteq \{\text{CHECK}\} \quad I, \pi^\triangleright \models v:t \Downarrow \odot, P}{I, \pi \models v:\{x:t \mid \langle e, E \rangle\} \Downarrow \odot, (\pi \mapsto \text{CHECK}) (\text{EVAL}); P} \text{CHECKREFINEMENTTYPE}$$

$$\frac{I(\pi) \subseteq \{\text{EVAL}\} \quad I, \pi^\triangleright, E\{x \mapsto v\} \models e \Rightarrow \langle \text{false}, s \rangle, \pi', P}{I, \pi \models v:\{x:t \mid \langle e, E \rangle\} \Downarrow \blacktimes, (\pi \mapsto \text{EVAL}) (\text{CHECK}); P; \underline{\text{not } s}} \text{REFUTEREFINEMENTPREDICATE}$$

$$\frac{I(\pi) \subseteq \{\text{EVAL}\} \quad I, \pi^\triangleright, E\{x \mapsto v\} \models e \Rightarrow \langle \text{true}, s \rangle, \pi', P}{I, \pi \models v:\{x:t \mid \langle e, E \rangle\} \Downarrow \checkmark, (\pi \mapsto \text{EVAL}) (\text{CHECK}); P; s} \text{CONFIRMREFINEMENTPREDICATE}$$

Figure 119: Refinement Type Checkers